

Segundo Objetivo: Construir un modelo computacional tejido específico del metabolismo de hoja de *S. tuberosum* (grupo Phureja) a partir de datos transcriptómicos de la variedad Colombia.

Archivos de referencia:

En el siguiente enlace se encuentran los ids de gen de DM8.1 <http://www.bioinformatics-lab.cn/pubs/dm8/>. En el siguiente enlace se encuentra la información de DM6.1 https://spuddb.uga.edu/dm_v6_1_download.shtml. El repositorio de Kelly Botero se encuentra en el siguiente enlace: <https://github.com/kellybotero/PotatoRecon/tree/master>. El repositorio del modelo botara se encuentra en el siguiente enlace: <https://github.com/ampinzonv/PstKB/tree/dev/Code>

Estructura general de botara_250214.xml (GEM de referencia)

Para describir la estructura general del GEM **botara_250214.xml** se usó el programa de MATLAB COBRA Constraint-Based Reconstruction and Analysis (COBRA) Toolbox. Las instrucciones de la página de COBRA Toolbox indican descargar el directorio de GitHub, lo cual se puede hacer con los comandos **git clone --depth=1** <https://github.com/opencobra/cobratoolbox.git> **cobratoolbox**, donde: el comando **git clone** le indica al sistema de control de versiones Git que copie un repositorio, la opción **--depth=1** especifica al Git que copie solo el commit más reciente, es decir la versión más reciente de un proyecto, el argumento <https://github.com/opencobra/cobratoolbox.git> especifica el enlace del directorio del proyecto a copiar y el argumento **cobratoolbox** especifica el nombre de la carpeta en la que se almacenará el directorio a copiar. En caso de tener algún problema con la copia de submódulos, se puede ejecutar el comando **git submodule update --init --recursive**, donde: el comando **git submodule** llama a la herramienta Git de manejo de repositorios anidados (submódulos), el comando **update** actualiza cada submódulo, la opción **--init** inicializa los submódulos si estos no han sido configurados y la opción **--recursive** aplica el proceso a cualquier submódulo existente dentro de los submódulos del directorio.

```
gibbs@gibbs:~/MATLABdocker$ git clone --depth=1 https://github.com/opencobra/cobratoolbox.git
```

1. Preparación de Docker de MATLAB para uso de COBRA Toolbox

Usando el comando **tmux attach-session -t 0** se accede a la sesión **0** de tmux en la cual se puede ejecutar comandos directamente en el servidor, es decir que son independientes de la conexión entre la máquina local y el servidor.

```
(base) gibbs@gibbs:~/analysis2.0/QualityControl/RawDataQC$ tmux attach-session -t 0
[detached (from session 0)]
```

Luego para iniciar MATLAB en un docker se ejecutó el comando **sudo docker run -it --rm --shm-size=13g --cpus="18" -v /home/gibbs/MATLABdocker/cobratoolbox:/home/matlab/Documents/MATLAB**

mathworks/matlab:r2022a /bin/bash -c "sudo apt-get update && sudo apt-get install -y git curl && matlab -nodisplay", donde:

- + El comando **sudo** (significa superuser do o substitute user do) permite al usuario ejecutar comandos con permisos administrativos.
- + El comando **docker run** es el comando base para iniciar o crear un contenedor Docker a partir de una imagen.
- + La opción **-it** viene de la unión entre **-i** (interactive) e **-t** (tty), donde **-i** mantiene abierto el STDIN, lo que permite al usuario interactuar con el contenedor. Mientras que **-t** asigna una pseudo-terminal a la interacción usuario contenedor.
- + La opción **--rm** remueve automáticamente la existencia del contenedor al cerrarlo.
- + La opción **--cpus="18"** define el tamaño de núcleos del contenedor en **18**.
- + La opción **--shm-size=13g** define el tamaño de **/dev/shm**, es decir la memoria compartida por partición en el contenedor y la ajusta a **13g** debido a que MATLAB (y otras aplicaciones) requieren de tamaños de memoria mayores al tamaño por defecto que asigna Docker de **64MB**.
- + La opción **-v** (volume mount) conecta un directorio de la máquina hospedero al contenedor, siendo en este caso la ruta del hospedero: **/home/gibbs/MATLABdocker/cobratoolbox** (ruta en la máquina local). Mientras que la ruta del directorio en el contenedor sería: **/home/matlab/Documents/MATLAB** (ruta en el contenedor). Esta conexión permite almacenar cambios generados por el contenedor en la máquina hospedera (específicamente el directorio definido). Es decir que permite compartir archivos entre el hospedero y el contenedor.
- + El argumento **mathworks/matlab:r2022a** especifica la imagen Docker a usar, en este caso en una imagen Docker oficial de MATLAB proveniente de MathWorks específicamente la versión **R2022a**.
- + Finalmente, el comando **/bin/bash -c "sudo apt-get update && sudo apt-get install -y git curl && matlab -nodisplay"** sobre escribe el comando por defecto del contenedor con un script de shell particular, en este caso: **/bin/bash** inicia una shell Bash; **-c "..."** ejecuta los comandos especificados dentro de la shell; **sudo apt-get update** actualiza la lista de paquetes (aquellas requeridas antes de instalar nuevos paquetes); **sudo apt-get install -y git curl** instala **git** y **curl** (pues son dependencias para las toolboxes de MATLAB); **matlab -nodisplay** inicia MATLAB en un modo no GUI (sin interfaz gráfica, adecuado para scripting).

NOTA: El directorio que se carga como volumen (con la opción **-v** (volume mount)) siempre se mantiene en comunicación entre el hospedero y el docker, lo que quiere decir que los cambios que se hagan desde el hospedero en el directorio cargado como volumen se observan simultáneamente en la pseudo terminal del docker.

La ejecución del comando **sudo docker run -it --rm --shm-size=13g --cpus="18" -v /home/gibbs/MATLABdocker/cobratoolbox:/home/matlab/Documents/MATLAB mathworks/matlab:r2022a /bin/bash -c "sudo apt-get update && sudo apt-get install -y git**

curl && matlab -nodisplay", construye un contenedor docker con las especificaciones dadas y despliega una pseudo consola.

Para añadir el directorio de **Cobra Toolbox** (que tiene implantado el directorio **E2F**) en la pseudo consola de MATLAB, se deben ejecutar los comandos **addpath(genpath('/home/matlab/Documents/MATLAB'))**; y **savepath**; . Donde la función de MATLAB **addpath()** añade una ruta específica a la ruta de búsqueda de MATLAB, lo que permite que las funciones y scripts en esos folders sean accesibles para MATLAB desde cualquier ubicación. Adicionalmente la función **genpath()** incluye un directorio específico y todos sus subdirectorios evitando carpetas especiales como **private**, **@class** o **+namespace**. Por otro lado, la función de MATLAB **savepath**; almacena el estado actual de las rutas de búsqueda de MATLAB en el archivo **pathdef.m**, este archivo se carga automáticamente al iniciar, lo que permite a MATLAB recordar rutas específicas.

Luego de añadir y guardar la ruta y directorio de interés, se ejecuta el comando **initCobraToolbox()** para confirmar el correcto funcionamiento de COBRA Toolbox. Luego para importar con COBRA Toolbox el modelo **botara_250214.xml** a MATLAB se ejecuta el comando **model = readCbModel('botara_250214.xml')**; (usualmente en formato **.xml** o **.m**).

```
>> addpath(genpath('/home/matlab/Documents/MATLAB'));
>> savepath;

>> initCobraToolbox;

  COBRA | COntstraint-Based Reconstruction and Analysis
        | The COBRA Toolbox - 2025
        |
        | Documentation:
        | http://opencobra.github.io/cobratoolbox

>> model = readCbModel('botara_250214.xml');
One subSystem per reaction, so each model.subSystems{x} is a character array.
```

Con el modelo **botara_250214.xml** ya importado se pueden revisar superficialmente todos sus campos con el comando **disp(fieldnames(model))**; , donde: la función **disp()** imprime en pantalla el contenido de lo que cubran sus paréntesis y la función **fieldnames(model)** retorna un arreglo de cadenas cada una representando el nombre de un campo en la estructura de un archivo o variable, en este caso la variable **model**. El comando indicó que para **model** existen los campos: **S**, **mets**, **b**, **csense**, **rxns**, **lb**, **ub**, **c**, **osenseStr**, **genes**, **rules**, **geneNames**, **compNames**, **comps**, **proteins**, **metNames**, **rxnNames**, **description**, **modelVersion** y **modelID**. Asimismo, si se quiere observar un resumen de la estructura del modelo, se puede retirar la función **fieldnames()** y ejecutar únicamente el comando **disp(model)**; , el cual muestra todos los campos del modelo, su tipo de variable y estructura.

```

>> disp(fieldnames(model));
{'S' }
{'mets' }
{'b' }
{'csense' }
{'rxns' }
{'lb' }
{'ub' }
{'c' }
{'osenseStr' }
{'genes' }
{'rules' }
{'geneNames' }
{'compNames' }
{'comps' }
{'proteins' }
{'metNames' }
{'rxnNames' }
{'description' }
{'modelVersion' }
{'modelID' }

>> disp(model);
S: [1940x2064 double]
mets: {1940x1 cell}
b: [1940x1 double]
csense: [1940x1 char]
rxns: {2064x1 cell}
lb: [2064x1 double]
ub: [2064x1 double]
c: [2064x1 double]
osenseStr: 'max'
genes: {2751x1 cell}
rules: {2064x1 cell}
geneNames: {2751x1 cell}
compNames: {2x1 cell}
comps: {2x1 cell}
proteins: {2751x1 cell}
metNames: {1940x1 cell}
rxnNames: {2064x1 cell}
description: 'botara_250214.xml'
modelVersion: [1x1 struct]
modelID: 'model - Wed Jun 14 12:28:21 2017'

```

NOTA: los campos usuales para almacenar información genética básica son **model.genes** (ubicación más común para arreglos de gene IDs), **model.geneNames** (campo opcional con los nombres de los genes), **model.geneMiriams** (campo opcional con referencias cruzadas entre bases de datos)

NOTA: los campos usuales para almacenar asociaciones Gen-reacción son **model.rxnGeneMat** (matriz de mapeo entre reacciones y genes), **model.grRules** (reglas Gen-reacción en formato de texto), **model.rules** (reglas Gen-reacción en formato computable)

De esta forma se observa que **model** almacena en formato computable las reglas Gen-reacción en el campo **rules**, asimismo se puede verificar que el **rules** sigue el formato '**x(geneID)**' (un formato común para codificar asociaciones genéticas en modelos metabólicos) con el script:

```

if isfield(model, 'rules') && any(cellfun(@(x) contains(x, 'x('), model.rules))
    fprintf('Model contains gene rules in model.rules\n');
end

```

Donde: la función **isfield(model, 'rules')** evalúa que exista el campo **rules** en la estructura de **model**, la función **any()** regresa verdadero si cada elemento del resultado de **cellfun()** es verdadero, la función **cellfun(@(x) contains(x, 'x('), model.rules)** aplica a cada celda del campo **rules** de **model** (**model.rules**) una función anónima **@(x)**, la cual toma la entrada **x** y aplica la función **contains(x, 'x(')**, pues esta evalúa si la cadena de entrada **x** contiene la subcadena exacta '**x('**', es decir que busca variables que inicien con **x** y se indexen como **x(1)**, **x(2)**, **x(3)**...

```
>> if isfield(model, 'rules') && any(cellfun(@(x) contains(x, 'x('), model.rules))
    fprintf('Model contains gene rules in model.rules\n');
end
Model contains gene rules in model.rules
>> 
```

Aunque **model** no presenta un campo como **model.rxnGeneMat** para almacenar las reglas Gen-reacción en una matriz de mapeo entre reacciones y genes, se puede validar la cantidad de asociaciones gen reacción con los campos **rules** y **genes** ejecutando el script:

```
% Definición de la variable nAssociatedRxns
if isfield(model, 'rules') && any(cellfun(@(x) ~isempty(x), model.rules))
    % Esta línea debe anteceder la de fprintf
    nAssociatedRxns = sum(cellfun(@(x) ~isempty(x), model.rules));
    % Ahora se puede usar la variable nAssociatedRxns
    fprintf('Gene-reaction rules exist for %d/%d reactions (%.1f%%)\n', ...
        nAssociatedRxns, length(model.rules), ...
        100*nAssociatedRxns/length(model.rules));

    % Extraer todos los genes únicos que se usen en las reglas
    usedGenes = unique(regexp([model.rules{:}], 'x\\(\\d+\\)', 'match'));
    nUsedGenes = length(usedGenes);
    fprintf('%d/%d genes are actually used in reaction rules\n', ...
        nUsedGenes, length(model.genes));
end
```

Donde el comando **~isempty(x)** retorna verdadero si **x** no está vacío.

```
>> % Definición de la variable nAssociatedRxns
>> if isfield(model, 'rules') && any(cellfun(@(x) ~isempty(x), model.rules))
    % Esta línea debe anteceder la de fprintf
    nAssociatedRxns = sum(cellfun(@(x) ~isempty(x), model.rules));

    % Ahora se puede usar la variable nAssociatedRxns
    fprintf('Gene-reaction rules exist for %d/%d reactions (%.1f%%)\n', ...
        nAssociatedRxns, length(model.rules), ...
        100*nAssociatedRxns/length(model.rules));

    % Extraer todos los genes únicos que se usen en las reglas
    usedGenes = unique(regexp([model.rules{:}], 'x\\(\\d+\\)', 'match'));
    nUsedGenes = length(usedGenes);
    fprintf('%d/%d genes are actually used in reaction rules\n', ...
        nUsedGenes, length(model.genes));

end
Gene-reaction rules exist for 1765/2064 reactions (85.5%)
2751/2751 genes are actually used in reaction rules
```

De un total de 2064 reacciones, 1765 están asociadas a reglas de gen reacción, por otro lado, de un total de 2751 genes del modelo 2751 se asocian a reglas de gen reacción.

Generación de archivo multifasta con los gene ID adecuados y mapeo de gene ID DM8 a NCBI

En este caso el modelo **botara_250214.xml** usa los gene ID del NCBI, mientras que los datos de expresión con los que se desea ajustar el modelo usan gen ID del DM8.1. Por lo tanto, con los gene ID del NCBI extraídos del modelo **botara_250214.xml** se puede generar un archivo multifasta que presente para cada ID del modelo la respectiva secuencia de aminoácidos (**botara_gene.pep.fasta**) o en su defecto de nucleótidos (**botara_gene.cdc.fasta**). Luego, se puede generar un archivo que designe los homólogos entre gene IDs del DM8.1 y gene IDs del NCBI presentes en el modelo **botara_250214.xml** con un BLAST 1 HSP entre el archivo **botara_gene.pep.fasta** o **botara_gene.cdc.fasta** y el archivo **DM8.1_gene.pep.fasta.gz** (<http://www.bioinformatics-lab.cn/pubs/dm8/> multifasta con las secuencias de aminoácidos pertenecientes a cada gene ID del DM8.1) o **DM8.1_gene.cdc.fasta.gz** (multifasta con las secuencias nucleótidos pertenecientes a cada gene ID del DM8.1). Finalmente, con el archivo de homólogos se pueden mapear los gene ID DM8.1 de los archivos de expresión a gene ID del NCBI presentes en **botara_250214.xml** para ejecutar Exp 2 Flux usando como entrada el modelo **botara_250214.xml** y los archivos de expresión con los gene ID mapeados.

Ya confirmado el almacenamiento de los id de genes en el modelo, se procede a volver a cargar -en caso de haber reiniciado la consola- el modelo SBML en formato **.xml** con el comando **model = readCbModel('potato_model.xml');** (para nuevas versiones de COBRA puede que se utilice el comando **model = readSBML('potato_model.xml', 1000);** donde 1000 es la condición por defecto). Luego, se extraen en la variable **gene_ids** los ids de genes con el comando **gene_ids = model.genes;** y se procede a guardar los ids de genes en un archivo de texto **potato_gene_ids.txt** con el script:

```
fid = fopen('potato_gene_ids.txt', 'w');
fprintf(fid, '%s\n', gene_ids{:});
fclose(fid);
disp('Gene IDs saved to potato_gene_ids.txt');
```

```
>> gene_ids = model.genes;
```

```
>> fid = fopen('potato_gene_ids.txt', 'w');
>> fprintf(fid, '%s\n', gene_ids{:});
>> fclose(fid);
>> disp('Gene IDs saved to potato_gene_ids.txt');
Gene IDs saved to potato_gene_ids.txt
>> ls
E2F          binary          documentation    jenkins-test.txt    runTestsAndGenerateReport.m  tutorials
Jenkinsfile  botara_250214.xml external         mi_archivo.txt      run_coverage_tests.m
LICENSE.md   codecov.yml       gene_list.txt     papers              src
README.rst   deprecated        initCobraToolbox.m potato_gene_ids.txt  test
```

Ya guardados los ids de genes en el archivo de texto **potato_gene_ids.txt** se cerró la pseudo consola y desde la consola de la máquina se continuó la generación del archivo multifasta con los gene ID adecuados y su respectiva secuencia en aminoácidos o nucleótidos.

Se instaló la dependencia pandas con el comando **pip install pandas openpyxl**

```
(base) gibbs@gibbs:~/MATLABdocker/cobratoolbox$ pip install pandas openpyxl
Requirement already satisfied: pandas in /home/gibbs/anaconda3/lib/python3.12/site-packages (2.2.2)
Requirement already satisfied: openpyxl in /home/gibbs/anaconda3/lib/python3.12/site-packages (3.1.5)
Requirement already satisfied: numpy>=1.26.0 in /home/gibbs/anaconda3/lib/python3.12/site-packages (from pandas) (1.26.4)
Requirement already satisfied: python-dateutil>=2.8.2 in /home/gibbs/anaconda3/lib/python3.12/site-packages (from pandas) (2.9.0.post0)
Requirement already satisfied: pytz>=2020.1 in /home/gibbs/anaconda3/lib/python3.12/site-packages (from pandas) (2024.1)
Requirement already satisfied: tzdata>=2022.7 in /home/gibbs/anaconda3/lib/python3.12/site-packages (from pandas) (2023.3)
Requirement already satisfied: et-xmlfile in /home/gibbs/anaconda3/lib/python3.12/site-packages (from openpyxl) (1.1.0)
Requirement already satisfied: six>=1.5 in /home/gibbs/anaconda3/lib/python3.12/site-packages (from python-dateutil>=2.8.2->pandas) (1.16.0)
```

Se verificó que python estuviese instalado con el comando **python**

```
(base) gibbs@gibbs:~/tools$ python
Python 3.12.2 | packaged by conda-forge | (main, Feb 16 2024, 20:50:58) [GCC 12.3.0] on linux
Type "help", "copyright", "credits" or "license" for more information.
>>> █
```

Se instaló requests (para llamar APIs) con el comando **pip install requests**

```
(base) gibbs@gibbs:~/tools$ pip install requests
Requirement already satisfied: requests in /home/gibbs/anaconda3/lib/python3.12/site-packages (2.32.3)
Requirement already satisfied: charset-normalizer<4,>=2 in /home/gibbs/anaconda3/lib/python3.12/site-packages (from requests) (3.3.2)
Requirement already satisfied: idna<4,>=2.5 in /home/gibbs/anaconda3/lib/python3.12/site-packages (from requests) (3.7)
Requirement already satisfied: urllib3<3,>=1.21.1 in /home/gibbs/anaconda3/lib/python3.12/site-packages (from requests) (2.2.3)
Requirement already satisfied: certifi>=2017.4.17 in /home/gibbs/anaconda3/lib/python3.12/site-packages (from requests) (2025.1.31)
```

Se resaltó que el archivo **potato_gene_ids.txt** contiene cuatro gene IDs del NCBI obsoletos:

LOC102577880 fructose-bisphosphate aldolase-like protein [*Solanum tuberosum* (potato)]

Gene ID: 102577880, discontinued on 2-Nov-2024

⚠ This record was replaced with [Gene ID: 102577667](#)

CGS1 cystathionine-gamma-synthase [*Solanum tuberosum* (potato)]

Gene ID: 102577457, discontinued on 8-Dec-2016

⚠ This record was replaced with [Gene ID: 102601571](#)

LOC102577487 dihydroflavonol 4-reductase [*Solanum tuberosum* (potato)]

Gene ID: 102577487, discontinued on 1-Nov-2024

⚠ This record was replaced with [Gene ID: 102587107](#)

LOC102577696 anthocyanidin synthase [*Solanum tuberosum* (potato)]

Gene ID: 102577696, discontinued on 31-Oct-2024

⚠ This record was replaced with [Gene ID: 102598323](#)

Asimismo, se resaltó que los cuatro gene IDs del NCBI obsoletos no presentan relación con IDs de la base de datos **Protein**, pero si con **Identical Protein Groups**, que también contiene secuencias proteicas:

Proteins	
Conserved Domains	0
Identical Protein Groups	1
Protein	0

Debido a lo anterior, se creó un script que:

- + **Cumple los requisitos de la API y servidores del NCBI:** solicita un email para acceder a la API del NCBI e implementa un límite de solicitudes por segundo (por defecto de 2 solicitudes/sec, pues el límite del NCBI es de 3/sec).
- + **Presenta características de robustez:** implementa reintentos automáticos con backoff exponencial. Evalúa ID obsoletos (manualmente y vía API). Asimismo, registra errores y el progreso de la ejecución del script.
- + **Facilita su ejecución por línea de comandos:** implementa el parseo de argumentos por línea de comandos y visualiza el progreso con estadísticas. Ayuda a solucionar problemas con un modo Debug.
- + **Mantiene la integridad de los datos:** permite preservar los Gene IDs originales en la salida y valida entradas y salidas.

```
#!/usr/bin/env python3
```

```
# #! Esta es la shebang que indica que el resto de líneas definen el interpretador.
```

```
"""
```

Propósito: buscar en el NCBI secuencias proteicas (en Protein o Identical Protein Groups) usando los Gene IDs de un archivo. Se incluye el monitoreo de progreso, testeo de API del NCBI, preservación del gene ID original, manejo apropiado de IDs obsoletos y límite de búsquedas por segundo

```
"""
```

```
# Importación de librerías estándar
```

```
import time          # Para limitar y retrasar solicitudes
```

```
from datetime import datetime # Para operaciones con marcas de tiempo
```

```
import argparse      # Para parsear argumentos por línea de comandos
```

```
import os            # Para operaciones con archivos (usualmente útil)
```

```
import sys          # Para resultados de operaciones/stderr a nivel de sistema
```

```
import traceback     # Para detallar el reporte de errores
```

```
# Importaciones de tercer partido
```

```
from Bio import Entrez  # Para programar utilidades del NCBI Entrez
```

```
from Bio import SeqIO  # Para manejar registros de secuencias en FASTA
```

```
#Las librerías estándar brindan las funcionalidades centrales del script
```

```
#Las importaciones de tercer partido de Biopython Entrez y SeqIO manejan las interacciones con el NCBI y el procesamiento de secuencias.
```

```
# Mapeo manual de gene IDs obsoletos conocidos y sus versiones actuales
```

```
# Esto actúa como un cache para evitar solicitudes API repetidas para IDs obsoletos conocidos
```

```
KNOWN_OBSOLETE_REPLACEMENTS = {
```

```
    '102577880': '102577667',
```

```
    '102577457': '102601571',
```

```
    '102577487': '102587107',
```

```
    '102577696': '102598323'
```



```
}
```

```
def test_ncbi_connection(email):
```

```
    """Prueba de conectividad a los servidores del NCBI
```

```
    Argumentos:
```

```
        email (str): dirección de correo a identificar por el NCBI (requerido por la API)
```

```
    Salida:
```

```
        bool: True si la conexión fue exitosa, de lo contrario False
```

```
    Particularidades de esta función:
```

1. Ajusta el email para el uso de la API del NCBI
2. Hace una solicitud simple de einfo para probar la conectividad
3. Brinda consejos si la conexión falla

```
    """
```

```
try:
```

```
    print("\nTesting NCBI connection...")
```

```
    Entrez.email = email # Requerido para el uso de la API del NCBI
```

```
    handle = Entrez.einfo() # Solicitud simple a una base de datos
```

```
    result = Entrez.read(handle) # Parsea la respuesta XML
```

```
    handle.close()
```

```
    print("✓ Connection successful")
```

```
    print(f"Available databases: {', '.join(result['DbList'][:5])}...") # Muestra 5 DBs
```

```
    return True
```

```
except Exception as e:
```

```
    print(f"\n✗ NCBI Connection Failed:", file=sys.stderr)
```

```
    print(f"Error: {str(e)}", file=sys.stderr)
```

```
    print("\nTroubleshooting tips:")
```

```
    print("1. Check your internet connection")
```

```
    print("2. Verify firewall isn't blocking HTTPS connections")
```

```
    print("3. Try again later if NCBI is down")
```

```
    return False
```

```
def parse_arguments():
```

```
    """Parseo y validación de argumentos en la línea de comandos
```

```
    Salida:
```

```
        argparse.Namespace: objeto contenedor de los argumentos parseados
```

```
    Particularidades de la función:
```

1. Ajusta los argumentos a parsear con una descripción y formato de ayuda
2. Define todos los posibles argumentos de línea de comandos
3. Regresa los argumentos parseados con opciones por defecto

```
    """
```

```
parser = argparse.ArgumentParser(
```

```

        description='Fetch protein sequences from NCBI Gene IDs',
        formatter_class=argparse.ArgumentDefaultsHelpFormatter # Muestra valores por
defecto en la ayuda
    )

```

```

# Argumentos requeridos
    parser.add_argument('-i', '--input', required=True, help='Input file with Gene IDs (one
per line)')
    parser.add_argument('-o', '--output', required=True, help='Output FASTA file path')
    parser.add_argument('-e', '--email', required=True, help='Email for NCBI (required by
their API)')

```

```

# Argumentos opcionales y opciones por defecto
    parser.add_argument('-r', '--rate', type=float, default=2,
        help='Max requests per second (NCBI limit=3, so default is conservative)')
    parser.add_argument('--max_retries', type=int, default=3,
        help='Max retry attempts for failed requests')
    parser.add_argument('--retry_delay', type=float, default=5,
        help='Base delay in seconds between retries (increases with each attempt)')

```

```

# Indicadores Booleanos (store_true actions)
    parser.add_argument('-d', '--description', action='store_true',
        help='Include original Gene ID in FASTA description line')
    parser.add_argument('--debug', action='store_true',
        help='Enable verbose debugging output')
    parser.add_argument('--check_obsolete', action='store_true',
        help='Check for obsolete Gene IDs and find current versions')

```

```

return parser.parse_args()

```

```

def fetch_with_retry(func, *args, **kwargs):
    """Enhanced retry wrapper with error logging and exponential backoff
Argumentos:
    func (callable): la función para reintentar
    *args: argumentos posicionales a pasar a la func
    **kwargs: palabras clave de argumentos:
        max_retries (int): número máximo de reintentos
        retry_delay (float): retraso entre intentos (aumenta con cada reintento)
        debug (bool): donde imprimir información de debug
Salida:
    Toda función exitosa
Subidas:
    Exception: si todas las solicitudes fallan, re-subir la última excepción
Se implementa:

```

1. Reintentos múltiples con aumento en retrasos
 2. Debug logging para cada intento
 3. Manejo de errores limpio
- """

```

max_retries = kwargs.pop('max_retries', 3)
retry_delay = kwargs.pop('retry_delay', 5)
debug = kwargs.pop('debug', False)
func_name = func.__name__

for attempt in range(max_retries):
    try:
        if debug:
            print(f"Attempt {attempt+1}/{max_retries}: {func_name}()")
        return func(*args, **kwargs)
    except Exception as e:
        if debug:
            print(f"✗ Attempt {attempt+1} failed:", file=sys.stderr)
            traceback.print_exc() # Seguimiento detallado de errores
        if attempt == max_retries - 1: # Último intento fallido
            raise # Re-sube la excepción
        # Backoff exponencial: espera larga después de cada falla
        time.sleep(retry_delay * (attempt + 1))

```

def fetch_protein_from_gene(gene_id, args, stats):

"""Búsqueda de secuencia proteica por gene ID y evaluación de obsolescencia
Argumentos:
gene_id (str): NCBI Gene ID a buscar
args (Namespace): arguments de línea de comandos
stats (dict): diccionario para seguir un conteo de éxitos/fallas
Salida:
Bio.SeqRecord.SeqRecord or None: secuencia Fasta a almacenar por éxito
Esta función maneja:

1. Evaluación de gene IDs obsoletos (si se activa)
2. Búsqueda de protein IDs relacionados
3. Almacenamiento de secuencias proteicas en formato FASTA
4. Preservación de ID original en la descripción si se solicita

"""

```

current_id = gene_id # Inicio con el ID original
is_obsolete = False # Registro de ID obsoleto encontrados

```

Evaluación de genes obsoletos si se activa en los argumentos

```

if args.check_obsolete:
    # Primero, evalúa el diccionario de mapeo
    if gene_id in KNOWN_OBSOLETE_REPLACEMENTS:
        current_id = KNOWN_OBSOLETE_REPLACEMENTS[gene_id]
        is_obsolete = True
        if args.debug:
            print(f" Using manual replacement: {gene_id} → {current_id}")

    # Si no se encuentra en el diccionario, evalúa usando la NCBI API
    else:
        try:
            # Query en el sistema NCBI history/link para encontrar reemplazos
            handle = Entrez.elink(dbfrom="gene", db="gene", id=gene_id,
cmd="neighbor_history")
            history = Entrez.read(handle)
            handle.close()

            # Evalua si hay enlaces de reemplazo
            if history and history[0]['LinkSetDbHistory']:
                for link in history[0]['LinkSetDbHistory']:
                    # 'gene_gene_repl' indica un ID de reemplazo
                    if link.get('LinkName') == 'gene_gene_repl' and link['Ids']:
                        current_id = link['Ids'][0] # Obtención del ID de reemplazo
                        is_obsolete = True
                        if args.debug:
                            print(f" Found replacement via API: {gene_id} → {current_id}")
                        break
        except Exception as e:
            if args.debug:
                print(f" Obsolete check failed for {gene_id}: {str(e)}")

# Buscar secuencias proteicas por el gene ID (posiblemente actualizado)
try:
    # Paso 1: obtener enlaces de la base de datos gene a protein
    handle = Entrez.elink(dbfrom="gene", db="protein", id=current_id)
    record = Entrez.read(handle)
    handle.close()

    # Evalúa si hay algún enlace de protein
    if not record or not record[0]['LinkSetDb']:
        if args.debug:
            print(f" No protein links found for {current_id}")
        return None

```

```

# Extrae todas los protein IDs relacionados
protein_ids = [link["Id"] for link in record[0]["LinkSetDb"][0]["Link"]]
if not protein_ids:
    if args.debug:
        print(f" Empty protein ID list for {current_id}")
    return None

# Paso 2: busca la primera protein sequence (en formato FASTA)
handle = Entrez.efetch(db="protein", id=protein_ids[0], rettype="fasta",
retmode="text")
seq_record = SeqIO.read(handle, "fasta") # Parsea FASTA
handle.close()

# Opcional: preservar el ID original en la descripción si se pide
if args.description:
    seq_record.description = f"{gene_id} {seq_record.description}"

if args.debug:
    print(f" Successfully fetched {seq_record.id}")

return seq_record

except Exception as e:
    if args.debug:
        print(f" Fetch failed for {current_id}: {str(e)}")
    return None

def process_gene_batch(gene_ids, args, stats):

    """Procesa un batch de gene IDs con rastreo de progreso y límite
    Argumentos:
        gene_ids (list): lista de gene IDs a procesar
        args (Namespace): argumentos de línea de comandos
        stats (dict): diccionario para rastrear conteo de éxitos/fallas
    Salida:
        list: lista de búsquedas exitosas de objetos SeqRecord
    Particularidades de la función:
        1. Implementa límite de solicitudes al NCBI
        2. Muestra actualizaciones de progreso
        3. Manejo de interrupciones de teclado con gratitud
        4. Rastrea estadísticas de éxitos/fallas
    """

    results = []

```

```

delay = 1 / args.rate # Calcula retraso entre solicitudes según límite

for i, gene_id in enumerate(gene_ids, 1): # Inicia conteo en 1
    try:
        # Visualización de progreso - actualización cada 10 IDs o en modo debug
        if i % 10 == 1 or args.debug:
            print(f"\rProcessing {i}/{len(gene_ids)}: {gene_id.ljust(15)}",
                  end="", flush=True)

            # Busca sequence con retry wrapper
            seq_record = fetch_with_retry(
                fetch_protein_from_gene,
                gene_id,
                args,
                stats,
                max_retries=args.max_retries,
                retry_delay=args.retry_delay,
                debug=args.debug
            )

            # Actualiza estadísticas basado en resultados
            if seq_record:
                results.append(seq_record)
                stats['success'] += 1
                if args.debug:
                    print(f"\n✓ Success: {gene_id} → {seq_record.id}")
            else:
                stats['failed'] += 1
                if args.debug:
                    print(f"\n✗ Failed: {gene_id}")

            time.sleep(delay) # Límite

    except KeyboardInterrupt:
        print("\nUser interrupted processing")
        break # Permite salida agradecida con Ctrl+C
    except Exception as e:
        if args.debug:
            print(f"\nError processing {gene_id}:", file=sys.stderr)
            traceback.print_exc()
            stats['failed'] += 1

print() # Línea nueva después de visualizar el progreso
return results

```

```

def main():
    """Función de ejecución central que coordina todos los pasos"""
    args = parse_arguments() # Parsea la línea de comandos

    # Verificación de la conectividad de la API antes de proceder
    if not test_ncbi_connection(args.email):
        sys.exit(1) # Exit si la conexión falla

    # Carga del archivo de entrada
    try:
        with open(args.input) as f:
            # Lectura de líneas no vacías, recortando espacios en blanco
            gene_ids = [line.strip() for line in f if line.strip()]

        print(f"\nLoaded {len(gene_ids)} Gene IDs from {args.input}")
        if args.debug:
            print("Sample IDs:", gene_ids[:5]) # Muestra los 5 primeros IDs en debug

    except Exception as e:
        print(f"Error loading input file: {str(e)}", file=sys.stderr)
        sys.exit(1)

    # Inicializa estadísticas
    stats = {'success': 0, 'failed': 0}
    print(f"\nStarting processing at {datetime.now().strftime('%H:%M:%S')}")
    print(f"Request rate: {args.rate}/sec")
    if args.description:
        print("Preserving original Gene IDs in output descriptions")

    try:
        # Procesa todos los IDs
        sequences = process_gene_batch(gene_ids, args, stats)

        # Almacena resultados si se encuentran sequences
        if sequences:
            SeqIO.write(sequences, args.output, "fasta")
            print(f"\nSaved {len(sequences)} sequences to {args.output}")

    # Reporte final
    print("\n" + "="*40)
    print(f"Processing completed at {datetime.now().strftime('%H:%M:%S')}")
    print(f"Success: {stats['success']}")
    print(f"Failed: {stats['failed']}")

```



```
print(f"Success rate: {stats['success']/len(gene_ids):.1%}") # Porcentaje con 1 decimal
```

```
except Exception as e:
```

```
    print(f"\nFatal error during processing: {str(e)}", file=sys.stderr)
```

```
    if args.debug:
```

```
        traceback.print_exc() # Traceback completo en modo debug
```

```
    sys.exit(1)
```

```
if __name__ == "__main__":
```

```
    main() # Punto de entrada al ejecutar el script
```

Funciones centrales del script:

La función **test_ncbi_connection(email)** evalúa la conectividad con los servidores del NCBI: primero establece el email requerido para la API del NCBI, luego realiza una solicitud de información de canal para evaluar la conectividad, asimismo proporciona consejos de resolución de fallas en la conexión y finalmente regresa un estado booleano de éxito.

La función **parse_arguments()** configura y parsea argumentos de línea de comando: primero usa **argparse** con el formato de argumentos requeridos **input file**, **output file** e **email**, asimismo incluye argumentos opcionales como **rate limiting**, **retry behavior** y **flags**. Parsear significa analizar e interpretar información estructurada.

La función **fetch_with_retry(func, *args, **kwargs)** regresa un wrapper (una clase, función o código que encapsula otro componente como una clase o API para proporcionar una interfaz distinta o funcionalidad adicional) con un backoff (evita errores o mejora el uso de recursos) exponencial: primero toma el reintento y sus argumentos como una función, luego implementa reintentos máximos al aumentar los retrasos, asimismo provee depuración de fallas detallada.

La función **fetch_protein_from_gene(gene_id, args, stats)** recupera secuencias proteicas de un recurso específico (fetches): primero maneja los Gene IDs obsoletos (ya sea del cache or de la API), encuentra protein IDs relacionados a través de enlaces de Entrez, luego recupera la secuencia proteica en formato FASTA, opcionalmente preserva el Gene ID original en la descripción, finalmente regresa un objeto SeqRecord de ser exitosa la recuperación.

La función **process_gene_batch(gene_ids, args, stats)** procesa un batch (la ejecución de una serie de comandos o trabajos de forma no-interactiva y automatizada, usualmente en una secuencia) de Gene IDs: primero establece un retraso entre cada solicitud, luego despliega el proceso cada 10 IDs o en modo debug, asimismo supervisa las estadísticas de conteo de éxitos y fracasos.

La función **main()** coordina el flujo de ejecución del script:

Parsear los argumentos de la línea de comando
Evaluar la conexión con el NCBI (salir si falla)
Cargar los Gene ID de entrada desde el archivo seleccionado
Inicializar el seguimiento de estadísticas
Procesar todos los IDs con la función **process_gene_batch()**
Escribir secuencias exitosas en un archivo FASTA de salida
Generar un reporte de resumen final

Usando el comando **tmux attach-session -t 0** accedemos a la sesión 0 de tmux en la cual podemos ejecutar comandos directamente en el servidor, los cuales son inmunes a desconexiones entre la máquina local y el servidor. En la sesión de tmux se creó un archivo multifasta **BotaraNCBIgeneID.fasta** con los gene ID, protein ID y secuencias de aminoácidos a partir de los gene ID del archivo **potato_gene_ids.txt** y el script **run_gene_to_protein11.py**, ejecutandose el commando **python3 run_gene_to_protein11.py -i potato_gene_ids.txt -o BotaraNCBIgeneID.fasta --email lcastiblanco@unal.edu.co -d --check_obsolete --debug --rate 1**

```
GNU nano 7.2                                run_gene_to_protein11.py
#!/usr/bin/env python3
"""
Enhanced script with progress monitoring, API testing, and original ID p>
"""

Saved 2751 sequences to BotaraNCBIgeneID.fasta

=====
Processing completed at 01:01:40
Success: 2751
Failed: 0
Success rate: 100.0%

GNU nano 7.2                                BotaraNCBIgeneID.fasta
>XP_015158380.1 102585678 XP_015158380.1 glutamate dehydrogenase
MNALAATNRNFKLAARLLGLDSKLEQSLIPFREIKVECTIPKDDGTLASFVGFRVQHDN
ARGPMKGGIRYHPEVDPDEVNALAQLMTWKTAVANIPYGGAKGGIGCSPDLSISELERL
```

Se resaltó que los IDs del archivo multifasta **DM8.1_gene.pep.fasta** tienen decimales en el código numérico, mientras que los IDs de los archivos de expresión no.

```
Clon17R2extracted_file1.csv
DM8sca115G00100,0.359425
```

GNU nano 7.2 DM8.1_gene.pep.fasta

>DM8sca115G00100.1

Por lo tanto se creó un script llamado **DM8NoDecimals.py** para remover los decimales de los archivos de expresión.

```
#!/usr/bin/env python3
```

```
# Línea shebang: asegura que al ejecutar el script directamente sea con Python 3.
```

```
# Importación de módulos requeridos:
```

```
# - "re" para operaciones de expresiones regulares (identificación de coincidencias/sustituciones de patrones).
```

```
# - "argparse" para parsear argumentos de la línea de comandos.
```

```
import re
```

```
import argparse
```

```
def clean_fasta_ids(input_file, output_file):
```

```
    """
```

```
    Limpia los FASTA IDs al remover puntos y números decimales.
```

```
    Argumentos:
```

```
    input_file (str): ruta al archivo FASTA de entrada.
```

```
    output_file (str): ruta para guardar el archivo FASTA limpio.
```

```
    """
```

```
# Abre ambos archivos, entrada (modo lectura) y salida (modo escritura).
```

```
# La afirmación "with" asegura que los archivos se cierran correctamente después de las operaciones.
```

```
with open(input_file, 'r') as infile, open(output_file, 'w') as outfile:
```

```
    # Itera sobre cada línea del archivo de entrada.
```

```
    for line in infile:
```

```
        # Evalúa si la línea es un encabezado FASTA (inicia con '>').
```

```
        if line.startswith('>'):
```

```
            # Remueve los espacios sobrantes (newline), los puntos y decimales.
```

```
            # Ejemplo: ">gene.123" → ">gene"
```

```
            cleaned_id = re.sub(r'\.d+', '', line.strip())
```

```
            # Escribe el encabezado limpiado en el archivo de salida, seguido de una nueva línea o newline.
```

```
            outfile.write(cleaned_id + '\n')
```

```
        else:
```

```
            # Si la línea es una secuencia (no un encabezado), lo escribe sin modificarlo.
```

```
            outfile.write(line)
```

```

def main():
    """
    Maneja argumentos de línea de comando y ejecuta el proceso de limpieza del FASTA.
    """
    # Inicializa el parseo de argumentos con una descripción del propósito del script.
    parser = argparse.ArgumentParser(
        description="Clean FASTA IDs by removing dots and trailing numbers."
    )

    # Define los argumentos de línea de comando:
    # - `i/--input`: archivo FASTA de entrada (requerido).
    # - `o/--output`: ruta de archivo de salida (requerido).
    parser.add_argument(
        "-i", "--input",
        required=True, # Argumento mandatorio.
        help="Input FASTA file (e.g., DM8.1_gene.pep.fasta)"
    )
    parser.add_argument(
        "-o", "--output",
        required=True, # Argumento mandatorio.
        help="Output FASTA file (e.g., DM8.1_geneNopoint.pep.fasta)"
    )

    # Parsea los argumentos proporcionados por el usuario en la línea de comando.
    args = parser.parse_args()

    # Ejecuta la función de limpieza con las rutas proporcionadas de entrada/salida.
    clean_fasta_ids(args.input, args.output)
    # Imprime un mensaje de confirmación que indica que el proceso se completó con éxito.
    print(f"Cleaned FASTA file saved as '{args.output}'.")

# Evaluación de punto de entrada: asegura que la función "main()" se ejecuta solo cuando el
script se ejecuta directamente,
# no cuando se importa como módulo en otro script.
if __name__ == "__main__":
    main()

```

```
python DM8NoDecimals.py -i DM8.1_gene.pep.fasta -o DM8.1_geneNodecimal.pep.fasta
```

```
(base) gibbs@gibbs:~/MATLABdocker/cobratoolbox$ python DM8NoDecimals.py -
i DM8.1_gene.pep.fasta -o DM8.1_geneNodecimal.pep.fasta
Cleaned FASTA file saved as 'DM8.1_geneNodecimal.pep.fasta'.
```

```
GNU nano 7.2 DM8.1_geneNodecimal.pep.fasta
>DM8C01G00010
MAALSKVVATILLMLMLIFSTVEAKTCESPSRRFKGLCFNTWKCRFVCSTERFSSGHCNGLRHRCICTKQLD>
```

Por otro lado, se resaltó que los IDs del archivo multifasta **BotaraNCBIgeneID.fasta** son tanto gene IDs como protein IDs y para el blast solo se requieren los gene ID.

```
GNU nano 7.2 BotaraNCBIgeneID.fasta
>XP_015158380.1 102585678 XP_015158380.1 glutamate dehydrogenase
MNALAATNRNFKLAARLLGLDSKLEQSLLIPFREIKVECTIPKDDGTLASFVGFRVQHDN
ARGPMKGGIRYHPEVDPDEVNALAQLMTWKTAVANIPYGGAKGGIGCSPDLSISELERL
```

Por lo tanto se creó un script llamado **JustgeneID2.py** para mantener solo los gene ID del archivo multifasta **BotaraNCBIgeneID.fasta**.

```
#!/usr/bin/env python3
```

```
# Shebang line: asegura que al ejecutar directamente el script será con Python 3.
```

```
from Bio import SeqIO
import argparse
```

```
def extract_gene_ids(input_file, output_file):
```

```
    """
```

Extrae los gene IDs numéricos de los encabezados del FASTA (majeja números positivos y negativos).

Argumentos:

input_file (str): ruta al archivo FASTA de entrada.

output_file (str): ruta del archivo FASTA de salida.

```
    """
```

```
with open(output_file, "w") as out_handle:
```

```
    for record in SeqIO.parse(input_file, "fasta"):
```

```
        parts = record.description.split()
```

```
        if len(parts) >= 2:
```

```
            potential_id = parts[1] # Get the second field
```

```
            # Evalúa:
```

```
            # 1) Números positivos regulares (solo dígitos)
```

```
            # 2) Números negativos (iniciados con '-' seguidos por dígitos)
```

```
            if potential_id.isdigit() or (potential_id.startswith('-') and
potential_id[1:].isdigit()):
```

```
                record.id = potential_id
```

```
                record.description = ""
```

```
                SeqIO.write(record, out_handle, "fasta")
```

```
                continue
```

```

        print(f"Warning: No valid numeric ID found in header: {record.description}")

def main():
    """
    Parsea los argumentos de línea de comando y ejecuta la extracción de gene ID.
    """
    parser = argparse.ArgumentParser(
        description="Extract numeric gene IDs from FASTA headers (handles
positive/negative numbers)."
    )

    parser.add_argument(
        "-i", "--input",
        required=True,
        help="Input FASTA file (e.g., input.fasta)"
    )
    parser.add_argument(
        "-o", "--output",
        required=True,
        help="Output FASTA file (e.g., output.fasta)"
    )

    args = parser.parse_args()
    extract_gene_ids(args.input, args.output)
    print(f"Cleaned FASTA file saved as '{args.output}'.")

if __name__ == "__main__":
    main()

```

Se ejecutó el comando `python3 JustgeneID0.py -i BotaraNCBIgeneID.fasta -o BotaraNCBIJustgeneID.fasta` y se obtuvo el archivo solo con los gene ID.

```

(base) gibbs@gibbs:~/MATLABdocker/cobratoolbox$ python3 JustgeneID0.py -i
BotaraNCBIgeneID.fasta -o BotaraNCBIJustgeneID.fasta

Cleaned FASTA file saved as 'BotaraNCBIJustgeneID.fasta'.

```

```

GNU nano 7.2 BotaraNCBIJustgeneID.fasta
>102585678
MNALAATNRNFKLAARLLGLDSKLEQSLIPFREIKVECTIPKDDGTLASFVGFRVQHDN

```

En la sesión de tmux se ejecutó con la herramienta Diamond un BLASTP entre el archivo multifasta **DM8.1_geneNodecimal.pep.fasta** y **BotaraNCBIJustgeneID.fasta**. Diamond se instaló en conda (herramienta que se activa con el comando **conda activate**) con el comando

conda install bioconda::diamond. Para confirmar la instalación correcta en conda se puede ejecutar el comando **diamond help**.

```
(base) gibbs@gibbs:~/MATLABdocker/cobratoolbox$ tmux attach-session -t 0
(base) gibbs@gibbs:~/MATLABdocker/cobratoolbox$ conda install bioconda::diamond
Retrieving notices: done
Channels:
- conda-forge
- bioconda
- defaults
Platform: linux-64
Collecting package metadata (repodata.json): | 
Proceed ([y]/n)? y
Downloading and Extracting Packages:
Preparing transaction: done
Verifying transaction: done
Executing transaction: done

(base) gibbs@gibbs:~/MATLABdocker/cobratoolbox$ diamond help
diamond v2.1.10.164 (C) Max Planck Society for the Advancement of Science, Benjamin Buchfink, University of Tuebingen
Documentation, support and updates available at http://www.diamondsearch.org
Please cite: http://dx.doi.org/10.1038/s41592-021-01101-x Nature Methods (2021)

Syntax: diamond COMMAND [OPTIONS]

Commands:
makedb          Build DIAMOND database from a FASTA file
prepdb          Prepare BLAST database for use with Diamond
blastp          Align amino acid query sequences against a protein reference database
blastx          Align DNA query sequences against a protein reference database
cluster         Cluster protein sequences
linclust        Cluster protein sequences in linear time
realign         Realign clustered sequences against their centroids
recluster       Recompute clustering to fix errors
reassign        Reassign clustered sequences to the closest centroid
view            View DIAMOND alignment archive (DAA) formatted file
merge-daa       Merge DAA files
help            Produce help message
version         Display version information
getseq          Retrieve sequences from a DIAMOND database file
dbinfo          Print information about a DIAMOND database file
test            Run regression tests
makeidx         Make database index
greedy-vertex-cover Compute greedy vertex cover

Possible [OPTIONS] for COMMAND can be seen with syntax: diamond COMMAND

Online documentation at http://www.diamondsearch.org
```

Ya instalado Diamond en conda, se ejecutó un BLASTP entre el archivo multifasta **DM8.1_geneNodecimal.pep.fasta** y **BotaraNCBIJustgeneID.fasta** con el comando **diamond blastp --query BotaraNCBIJustgeneID.fasta --db DM8.1_geneNodecimal.pep.fasta --matrix blosum62 --id 40 --max-target-seqs 1 --outfmt 6 --out d_BLASTP_DM8XBotara.tsv --threads 8**, donde **diamond blastp** hace referencia a un alineamiento proteína proteína de la herramienta diamond, **--query** al archivo multifasta query, **--subject** al archivo multifasta target, **--matrix blosum62** al uso de una matriz de sustitución BLOSUM62, **--id 40** a que se reporta solo los hits de identidad $\geq 40\%$, **--max-target-seqs 1** a que se reporta solo el mejor HSP por hit del query en el target y **--out** al nombre del archivo de salida.

```
(base) gibbs@gibbs:~/MATLABdocker/cobratoolbox$ diamond blastp --query BotaraNCBIJustgeneID.fasta --db DM8.1_geneNodecimal.pep.fasta --matrix blosum62 --id 40 --max-target-seqs 1 --outfmt 6 --out d_BLASTP_DM8XBotara2.tsv --threads 8
diamond v2.1.10.164 (C) Max Planck Society for the Advancement of Science, Benjamin Buchfink, University of Tuebingen
Documentation, support and updates available at http://www.diamondsearch.org
Please cite: http://dx.doi.org/10.1038/s41592-021-01101-x Nature Methods (2021)

#CPU threads: 8
Scoring parameters: (Matrix=BLOSUM62 Lambda=0.267 K=0.041 Penalties=11/1)
Temporary directory:
#Target sequences to report alignments for: 1
Opening the database... [0.083s]
Database: DM8.1_geneNodecimal.pep.fasta (type: FASTA file, sequences: 40957, letters: 14216148)
Block size = 2000000000
Opening the input file... [0.002s]
Opening the output file... [0s]
```

```
Total time = 1.615s
Reported 2748 pairwise alignments, 2748 HSPs.
2748 queries aligned.
```

GNU nano 7.2		d_BLASTP_DM8XBotara2.tsv									
102585678	DM8C10G29340	100	411	0	0	1	411	1	411	1.01e-308	833
102599312	DM8C03G14470	100	411	0	0	1	411	1	411	3.01e-310	837
102586580	DM8C05G22590	100	411	0	0	1	411	35	445	3.01e-307	830

Posterior al BLASTp, se creó un script llamado **FromDM8toBotara6.py** para mapear los gene ID de los archivos de expresión a los gene ID del NCBI.

FromDM8toBotara6.py

```
#!/usr/bin/env python3
```

```
"""
```

Script para mapear gene IDs DM8 a NCBI a través de un archivo BLASTP.

```
"""
```

```
import csv
import re
import argparse
```

```
def parse_arguments():
```

```
    """Parsea los argumentos de la línea de comando archivos de entrada/salida."""
```

```
    parser = argparse.ArgumentParser(
        description="Map internal gene IDs in an expression CSV file to NCBI gene IDs using
a BLASTP mapping file."
    )
    parser.add_argument(
        '-e', '--expression', required=True,
        help='Input expression CSV file (with internal gene IDs)'
    )
```



```

)
parser.add_argument(
    '-b', '--blast', required=True,
    help='Input BLASTP file (DM8C ID to NCBI gene ID, tab-delimited)'
)
parser.add_argument(
    '-o', '--output', default='expression_ncbi.csv',
    help='Output CSV file with NCBI gene IDs (default: expression_ncbi.csv)'
)
return parser.parse_args()

```

def main():

Parsear argumentos de la línea de comando

args = parse_arguments()

--- Paso 1: Parsear el BLASTP para mapear los IDs numeric DM8C a NCBI ---

blast_mapping = {}

with open(args.blast) as blast_file:

for line in blast_file:

parts = line.strip().split("\t") **#** Asume delimitación por tabulaciones

if len(parts) >= 12: **#** El archivo BLAST.tsv tiene 12 columnas

dm8c_id = parts[1] **#** La segunda columna es el ID DM8C (e.g., DM8C10G29340)

ncbi_gene_id = parts[0] **#** La primera columna es el ID NCBI (e.g., 102585678)

Estandarizar el formato DM8C ID

dm8c_key = re.sub(r"\.d+\$", "", dm8c_id).replace("DM8C", "DM8sca")

Mapear los ID DM8C a NCBI

blast_mapping[dm8c_key] = ncbi_gene_id

--- Paso 2: procesar archivo de expresión y escribir salida con los IDs del NCBI ---

with open(args.expression, newline="") as infile, open(args.output, 'w', newline=") as
outfile:

reader = csv.reader(infile)

writer = csv.writer(outfile)

Saltar encabezado del archivo de entrada y escribir un nuevo encabezado estandarizado

header = next(reader)

writer.writerow(["Gene", "Expression_Value"])

Procesar cada línea en el archivo de expresión

for row in reader:

if len(row) < 2: **#** saltar líneas incompletas

continue

```

gene_id, expression_value = row[0], row[1]

# Estandarizar los gene ID del archivo de expresión para que coincida con las claves de
mapeo
standardized_gene_id = re.sub(r'\.ld+$', '', gene_id).replace("DM8C", "DM8sca")

# Buscar los ID de NCBI en el diccionario de mapeo
ncbi_id = blast_mapping.get(standardized_gene_id)

if ncbi_id:
    # Escribe líneas con el ID de NCBI ID y valor de expresión
    writer.writerow([ncbi_id, expression_value])
else:
    print(f"Warning: No NCBI ID found for {gene_id} (standardized as
{standardized_gene_id})")

if __name__ == "__main__":
    main()

```

Se ejecutó el commando **python3 FromDm8toBotara6.py -e Clon17R2extracted_file1.csv -b d_BLASTP_DM8XBotara2.tsv -o Clon17ncbi6.csv**.

```
(base) gibbs@gibbs:~/MATLABdocker/cobratoolbox$ python3 FromDm8toBotara6.py -e Clon17R2extracted_file1.csv -b d_BLASTP_DM8XBotara2.tsv -o Clon17ncbi6.csv
```

```

GNU nano 7.2                                                                    Clon17ncbi6.csv
Gene,Expression_Value
102585405,0.000000
102603430,0.000000

```

python3 FromDm8toBotara6.py -e Clon39R2extracted_file2.csv -b d_BLASTP_DM8XBotara2.tsv -o Clon39ncbi6.csv

```
(base) gibbs@gibbs:~/MATLABdocker/cobratoolbox$ python3 FromDm8toBotara6.py -e Clon39R2extracted_file2.csv -b d_BLASTP_DM8XBotara2.tsv -o Clon39ncbi6.csv
```

```

GNU nano 7.2                                                                    Clon39ncbi6.csv
Gene,Expression_Value
102585405,0.000000
102603430,0.000000

```

python3 FromDm8toBotara6.py -e ClonColombiaR2extracted_file3.csv -b d_BLASTP_DM8XBotara2.tsv -o ClonColombiancbi6.csv

```
(base) gibbs@gibbs:~/MATLABdocker/cobratoolbox$ python3 FromDm8toBotara6.py -e ClonColombiaR2extracted_file3.csv -b d_BLASTP_DM8XBotara2.tsv -o ClonColombiancbi6.csv
```

```

GNU nano 7.2                                                                    ClonColombiancbi6.csv
Gene,Expression_Value
102585405,0.000000
102603430,0.000000

```

```
python3 FromDm8toBotara6.py -e FlorBlancaR2extracted_file4.csv -b d_BLASTP_DM8XBotara2.tsv -o FlorBlancancbi6.csv
```

```
(base) gibbs@gibbs:~/MATLABdocker/cobratoolbox$ python3 FromDm8toBotara6.py -e FlorBlancaR2extracted_file4.csv -b d_BLASTP_DM8XBotara2.tsv -o FlorBlancancbi6.csv
```

```
GNU nano 7.2 FlorBlancancbi6.csv
Gene,Expression_Value
102585405,0.000000
102603430,0.000000

(base) gibbs@gibbs:~/MATLABdocker/cobratoolbox$ scp Clon17ncbi6.csv E2F/E2F/
(base) gibbs@gibbs:~/MATLABdocker/cobratoolbox$ scp Clon39ncbi6.csv E2F/E2F/
(base) gibbs@gibbs:~/MATLABdocker/cobratoolbox$ scp ClonColombiancbi6.csv E2F/E2F/
(base) gibbs@gibbs:~/MATLABdocker/cobratoolbox$ scp FlorBlancancbi6.csv E2F/E2F/
```

Revisión de funciones Exp 2 Flux en MATLAB

Se usó como referencia el archivo **Mariabotara.m** en el cual las funciones resaltadas en amarillo provienen de archivos **.m** adicionales de Exp 2 Flux integrados en MATLAB:

dbstop if error % which line has an error

%% 1. Cargar modelo XML

```
modelo = readCbModel('botara_250214.xml');
modelo.rules = MariamapIndexToGeneRule(modelo);
modelo.rev = modelo.lb < 0;
modelo.lb_original = modelo.lb;
modelo.ub_original = modelo.ub;
```

%% 2. Cargar datos de expresión

```
exp_Flor = MariacargarExpresion2('TranscriptsExpCSVs/FlorBlancancbi6.csv');
exp_Colombia = MariacargarExpresion2('TranscriptsExpCSVs/ClonColombiancbi6.csv');
exp_Clon39 = MariacargarExpresion2('TranscriptsExpCSVs/Clon39ncbi6.csv');
exp_Clon17 = MariacargarExpresion2('TranscriptsExpCSVs/Clon17ncbi6.csv');
```

%% 3. Integrar expresión (usando método 'mean')

```
modelo_Flor = Mariaexp2flux(modelo, exp_Flor, [], [], 'mean', true);
modelo_Colombia = Mariaexp2flux(modelo, exp_Colombia, [], [], 'mean', true);
modelo_Clon39 = Mariaexp2flux(modelo, exp_Clon39, [], [], 'mean', true);
modelo_Clon17 = Mariaexp2flux(modelo, exp_Clon17, [], [], 'mean', true);
```

%% 4. Definir función objetivo: 'EX_Biomass[c]'

```
modelo_Flor = changeObjective(modelo_Flor, 'Ex_Biomass[c]');
modelo_Colombia = changeObjective(modelo_Colombia, 'Ex_Biomass[c]');
modelo_Clon39 = changeObjective(modelo_Clon39, 'Ex_Biomass[c]');
modelo_Clon17 = changeObjective(modelo_Clon17, 'Ex_Biomass[c]');
```

%% 5. Optimizar modelo

```
sol_Flor = optimizeCbModel(modelo_Flor);  
sol_Colombia = optimizeCbModel(modelo_Colombia);  
sol_Clon39 = optimizeCbModel(modelo_Clon39);  
sol_Clon17 = optimizeCbModel(modelo_Clon17);
```

%% 6. Mostrar resultados

```
fprintf('\nTasa de biomasa optimizada:\n');  
fprintf('Flor Blanca : %.4f\n', sol_Flor.f);  
fprintf('Clon Colombia : %.4f\n', sol_Colombia.f);  
fprintf('Clon 39 : %.4f\n', sol_Clon39.f);  
fprintf('Clon 17 : %.4f\n', sol_Clon17.f);
```

% Guardar tabla de reacciones con mayores cambios en límites

```
%Luiswritetable(top10Limites, 'top10_cambios_limites_reacciones_clones.csv');
```

% Guardar modelos integrados por condición/clon

```
save('modelo_FlorBlanca.mat', 'modelo_Flor');  
save('modelo_ClonColombia.mat', 'modelo_Colombia');  
save('modelo_Clon39.mat', 'modelo_Clon39');  
save('modelo_Clon17.mat', 'modelo_Clon17');
```

%% Comparar fenotipos metabólicos: flujos por reacción

% Crear tabla de flujos por reacción

```
tablaFenotipos = table(modelo.rxns, modelo.rxnNames, ...  
    sol_Flor.x, sol_Colombia.x, sol_Clon39.x, sol_Clon17.x, ...  
    'VariableNames', {'RxnID', 'Name', 'FlorBlanca', 'Colombia', 'Clon39', 'Clon17'});
```

% Calcular desviación estándar y rango por reacción

```
tablaFenotipos.std_flux = std([tablaFenotipos.FlorBlanca, ...  
    tablaFenotipos.Colombia, ...  
    tablaFenotipos.Clon39, ...  
    tablaFenotipos.Clon17], 0, 2);
```

```
tablaFenotipos.range_flux = max([tablaFenotipos.FlorBlanca, ...  
    tablaFenotipos.Colombia, ...  
    tablaFenotipos.Clon39, ...  
    tablaFenotipos.Clon17], [], 2) - ...  
min([tablaFenotipos.FlorBlanca, ...  
    tablaFenotipos.Colombia, ...  
    tablaFenotipos.Clon39, ...  
    tablaFenotipos.Clon17], [], 2);
```

```

% Ordenar por mayor variabilidad
tablaFenotiposOrdenada = sortrows(tablaFenotipos, 'range_flux', 'descend');

% Seleccionar top 20 reacciones más variables
top10Fenotipos = tablaFenotiposOrdenada(1:10, :);

% Mostrar top 20 en consola
disp(top10Fenotipos)

% Guardar tablas como CSV
saveTableAsCSV(tablaFenotipos, 'FenotipoMetabolico_Botara.csv');
saveTableAsCSV(top10Fenotipos, 'Top20_Reacciones_Fenotipo_Variable.csv');

```

```

GNU nano 7.2                                                                 Mariabotara.m
dbstop if error % which line has an error
%% 1. Cargar modelo XML
modelo = readCbModel('botara_250214.xml');
modelo.rules = MariamapIndexToGeneRule(modelo);
modelo.rev = modelo.lb < 0;
modelo.lb_original = modelo.lb;
modelo.ub_original = modelo.ub;

```

El archivo **MariamapIndexToGeneRule** contiene:

```

function geneRules = mapIndexToGeneRule(model)
    geneRules = model.rules;
    for i = 1:length(geneRules)
        rule = geneRules{i};
        if isempty(rule)
            continue
        end
        matches = regexp(rule, 'x\((\d+)\)', 'tokens');
        for j = 1:length(matches)
            idx = str2double(matches{j}{1});
            if idx <= length(model.genes)
                rule = strrep(rule, ['x(' matches{j}{1} ')'], model.genes{idx});
            end
        end
        geneRules{i} = rule;
    end
end

```

```

GNU nano 7.2 MariamapIndexToGeneRule.m
function geneRules = mapIndexToGeneRule(model)
    geneRules = model.rules;
    for i = 1:length(geneRules)
        rule = geneRules{i};
        if isempty(rule)
            continue
        end
        matches = regexp(rule, 'x\\((\\d+)\\)', 'tokens');
        for j = 1:length(matches)
            idx = str2double(matches{j}{1});
            if idx <= length(model.genes)
                rule = strrep(rule, ['x(' matches{j}{1} ')'], model.genes{idx});
            end
        end
        geneRules{i} = rule;
    end
end
end

```

El archivo **MariacargarExpresion2** contiene:

```

function expressionStruct = cargarExpresion2(expressionFile)
    % Cargar archivo con columnas: Gene, Expression_Value
    tabla = readtable(expressionFile);

    % Convertir a estructura con campos esperados por exp2flux
    expressionStruct.genes = cellstr(string(tabla.Gene)); % Lista de genes
    expressionStruct.data = tabla.Expression_Value(:); % Vector columna obligatorio
end

```

```

GNU nano 7.2 MariacargarExpresion2.m
function expressionStruct = cargarExpresion2(expressionFile)
    % Cargar archivo con columnas: Gene, Expression_Value
    tabla = readtable(expressionFile);

    % Convertir a estructura con campos esperados por exp2flux
    expressionStruct.genes = cellstr(string(tabla.Gene)); % Lista de genes
    expressionStruct.data = tabla.Expression_Value(:); % Vector columna obligatorio
end

```

El archivo **Mariaexp2flux** contiene:

```

function model = exp2flux(model, expression, organism, typeId, missing, scale)

% EXP2FLUX Convert gene expression data to FBA fluxes
%
% This function calculates the flux boundaries for each reaction in a
% metabolic model based on gene expression data and the associated GPR rules.
%
% INPUTS:
% model - COBRA model structure (must include rules and bounds).
% expression - A structure with two fields:
% * genes: Cell array of gene IDs corresponding to expression data.
% * data: Matrix where rows represent genes and columns represent samples.

```

```

% organism - (Optional) KEGG organism identifier for pathway data.
% typeID - (Optional) Type of gene IDs in GPR rules ('entrez' or 'kegg').
% missing - (Optional) Method for handling missing gene expression values.
%           Options: 'min', '1q', 'mean', 'median', '3q', 'max'. Default: 'mean'.
% scale - (Optional) Boolean indicating whether to scale expression data to
%          a maximum value of 1000. Default: false.
%
% OUTPUT:
% model - Modified COBRA model with updated flux bounds based on expression
data.
%
% USAGE:
% updatedModel = exp2flux(model, expression, 'hsa', 'entrez', 'mean', true);
%
% DEPENDENCIES:
% * Requires a function or API to fetch KEGG pathway data if organism/typeID is
provided.
% * COBRA Toolbox functions for model manipulation.
% VERSION:
% 1.0 - January 2025

% Default parameters
if nargin < 5
    missing = 'mean'; % Default missing value method
end
if nargin < 6
    scale = false; % Default scaling behavior
end

% Load KEGG pathway associations if organism and typeID are provided
if ~isempty(organism) && ~isempty(typeID)
    % Use a KEGG API equivalent for MATLAB (requires custom implementation)
    data = fetchKEGGPathways(organism, typeID);
    data = regexp(data, '\d+$', ''); % Remove numeric suffixes from pathway names
else
    data = [];
end

% Calculate expression for each GPR rule in the model
exp = calculateGPRExpression(model.rules, expression, data, missing);

% Scale expression data if specified
if scale
    exp = round((exp / max(exp, [], 'omitnan')), 6) * 1000;

```

```

end

% Adjust flux bounds based on expression data
model.lb = -exp; % Lower bounds
model.lb(~model.rev) = 0; % Ensure irreversible reactions have zero lower bound
model.ub = exp; % Upper bounds

% Preserve original bounds for exchange reactions
exchangeReactions = findExchReact(model);
model.lb(exchangeReactions) = model.lb_original(exchangeReactions);
model.ub(exchangeReactions) = model.ub_original(exchangeReactions);
end

function exp = calculateGPRExpression(grRules, expression, data, missing)
    % CALCATEGPREXPRESSION Evaluate GPR rules to calculate expression for each
    reaction.
    %
    % INPUTS:
    % grRules - Cell array of GPR rules for the model.
    % expression - Structure containing genes and expression data.
    % data - KEGG pathway associations (optional).
    % missing - Method for handling missing gene expression values.
    %
    % OUTPUT:
    % exp - Vector of expression values corresponding to reactions.

    numReactions = length(grRules);
    exp = zeros(numReactions, 1);

    for i = 1:numReactions
        gpr = grRules{i};
        if isempty(gpr)
            exp(i) = getMissingValue(expression, data, missing);
            continue;
        end

        % Parse GPR rule
        complexes = strsplit(regexprep(gpr, '[]', ''), ' or ');
        minValues = zeros(length(complexes), 1);

        for j = 1:length(complexes)
            genes = strsplit(complexes{j}, ' and ');
            geneExp = zeros(length(genes), 1);

```



```

    for k = 1:length(genes)
        gene = strtrim(genes{k});
        idx = find(strcmp(expression.genes, gene), 1);

        if ~isempty(idx)
            geneExp(k) = mean(expression.data(idx, :), 'omitnan');
        else
            geneExp(k) = getMissingValue(expression, data, missing);
        end
    end

    minValues(j) = min(geneExp);
end

exp(i) = sum(minValues, 'omitnan');
if exp(i) == 0
    exp(i) = getMissingValue(expression, data, missing);
end
end
end
function missingValue = getMissingValue(expression, data, method)
    % GETMISSINGVALUE Calculate a value for missing genes based on a specified
    method.
    %
    % INPUTS:
    % expression - Structure with gene expression data.
    % data       - KEGG pathway associations (optional).
    % method     - Method for handling missing values ('min', '1q', 'mean', etc.).
    %
    % OUTPUT:
    % missingValue - The calculated value for missing genes.

    allValues = mean(expression.data, 2, 'omitnan');
    switch method
        case 'min'
            missingValue = min(allValues);
        case '1q'
            missingValue = quantile(allValues, 0.25);
        case 'mean'
            missingValue = mean(allValues, 'omitnan');
        case 'median'
            missingValue = median(allValues, 'omitnan');
        case '3q'
            missingValue = quantile(allValues, 0.75);
    end
end

```

```

        case 'max'
            missingValue = max(allValues);
        otherwise
            error('Invalid method for missing values.');
```

end

end

```

function exchangeReactions = findExchReact(model)
    % FINDEXCHREACT Identify exchange reactions in a COBRA model.
    %
    % INPUT:
    %   model - COBRA model structure.
    %
    % OUTPUT:
    %   exchangeReactions - Indices of exchange reactions in the model.

    exchangeReactions = find(sum(abs(model.S), 1) == 1);
end

function data = fetchKEGGPathways(organism, typeId)
    % FETCHKEGGPATHWAYS Placeholder for fetching KEGG pathway data.
    %
    % INPUTS:
    %   organism - KEGG organism identifier.
    %   typeId   - Type of gene IDs ('entrez', 'kegg').
    %
    % OUTPUT:
    %   data     - Pathway associations (to be implemented).

    % This function requires custom implementation or use of a KEGG API for MATLAB.
    error('KEGG pathway fetching is not implemented.');
```

end

```

GNU nano 7.2                               Mariaexp2flux.m
function model = exp2flux(model, expression, organism, typeId, missing, scale)

    % EXP2FLUX Convert gene expression data to FBA fluxes
    %
    % This function calculates the flux boundaries for each reaction in a
    % metabolic model based on gene expression data and the associated GPR rules.
```

El archivo **changeObjective** contiene:

```

function model = changeObjective(model, rxns, objectiveCoeff)
% Changes the objective function of a constraint-based model
```

```

%
% USAGE:
%
% model = changeObjective(model, rxns, objectiveCoeff)
%
% INPUTS:
% model:      COBRA model structure
% rxns:       a string or a cell array of strings matching some model.rxns{i}
%
% OPTIONAL INPUT:
% objectiveCoeff: Value of objective coefficient for each reaction (Default = 1)
%
% OUTPUT:
% model:      COBRA model structure with new objective
%
% .. Author: - Monica Mo & Markus Herrgard - 8/21/06

if (nargin < 3)
    objectiveCoeff = 1;
end

rxnID = findRxnIDs(model,rxns);

model.c = zeros(size(model.c));

if iscell(rxns)
    missingRxns = rxns(rxnID == 0);
    for i = 1:length(missingRxns)
        fprintf('%s not in model\n',missingRxns{i});
    end
    rxnID = rxnID(rxnID ~= 0);
    if (length(objectiveCoeff) > 1)
        objectiveCoeff = objectiveCoeff(rxnID ~= 0);
    end
end

if (isempty(rxnID) | rxnID == 0)
    error('Objective reactions not found in model!');
else
    model.c(rxnID) = objectiveCoeff;
end

```

```

GNU nano 7.2                                changeObjective.m
function model = changeObjective(model, rxns, objectiveCoeff)
% Changes the objective function of a constraint-based model
%
% USAGE:
%
%     model = changeObjective(model, rxns, objectiveCoeff)
%

```

El archivo **optimizeCbModel** contiene:

```

function solution = optimizeCbModel(model, osenseStr, minNorm, allowLoops, param)
% Solves flux balance analysis problems, and variants thereof
%
% Solves LP problems of the form
%
% .. math::
% \begin{aligned}
% \max/\min \quad & \sim c^T v \\
% \text{s.t.} \quad & \sim S v = b \\
% & \sim C v \leq d \\
% & \sim lb \leq v \leq ub
% \end{aligned}
%
% Optionally, it also solves a second cardinality optimisation problem
%
% \begin{aligned}
% \max/\min \quad & \sim g1 \cdot |v|_1 \\
% \text{s.t.} \quad & \sim S v = b \\
% & \sim C v \leq d \\
% & \sim lb \leq v \leq ub \\
% & \sim c^T v = c^T v_{\text{Star}}
% \end{aligned}
%
% Optionally, it also solves a second QP problem
%
% \begin{aligned}
% \max/\min \quad & \sim g2 \cdot |v|_2 + 0.5 v^T F v \\
% \text{s.t.} \quad & \sim S v = b \\
% & \sim C v \leq d \\
% & \sim lb \leq v \leq ub \\
% & \sim c^T v = c^T v_{\text{Star}}
% \end{aligned}
%
% Optionally, it also solves a second cardinality optimisation problem
%
% \begin{aligned}
% \max/\min \quad & \sim g0 \cdot |v|_0 + g1 \cdot |v|_1 \\
% \text{s.t.} \quad & \sim S v = b \\
% & \sim C v \leq d \\
% & \sim lb \leq v \leq ub
% \end{aligned}

```

```

%      ~& c^T*v == c^T*vStar
%
% where vStar is the optimal solution to the first LP problem.
%
% USAGE:
%
% solution = optimizeCbModel(model, osenseStr, minNorm, allowLoops, param)
%
% INPUT:
% model:      (the following fields are required - others can be supplied)
%
%             * S - `m x n` Stoichiometric matrix
%             * c - `n x 1` Linear objective coefficients
%             * lb - `n x 1` Lower bounds on net flux
%             * ub - `n x 1` Upper bounds on net flux
%
% OPTIONAL INPUTS:
% model:
%
%             * b - `m x 1` change in concentration with time
%             * csense - `m x 1` character array with entries in {L,E,G}
%               (The code is backward compatible with an m + k x 1 csense vector,
%                where k is the number of coupling constraints)
%             * mets `m x 1` metabolite abbreviations
%
%             * C - `k x n` Left hand side of C*v <= d
%             * d - `k x 1` Right hand side of C*v <= d
%             * ctrs `k x 1` Cell Array of Strings giving IDs of the coupling constraints
%             * dsense - `k x 1` character array with entries in {L,E,G}
%
%             * `.evars` : evars x 1 Column Cell Array of Strings IDs of the additional
variables
%
%             * `.E` : n x evars The additional Variable Matrix
%             * `.evarub` : evars x 1 The upper bounds of the variables from E;
%             * `.evarlb` : evars x 1 The lower bounds of the variables from E;
%             * `.evarc` : evars x 1 The objective coefficients of the variables from E;
%             * `.D` : k x evars The matrix coupling additional Constraints (form C),
with additional Variables (from E);
%
%             * g0 - `n x 1` weights on zero norm, where positive is minimisation,
negative is maximisation, zero is neither.
%             * g1 - `n x 1` weights on one norm, where positive is minimisation,
negative is maximisation, zero is neither.
%             * g2 - `n x 1` weights on two norm
%

```

```

% osenseStr:      Maximize ('max')/minimize ('min') (opt, default =
%                'max') linear part of the objective. Nonlinear
%                parts of the objective are always assumed to be
%                minimised.
%
% minNorm:        {(0), 'one', 'zero', > 0 , n x 1 vector, 'optimizeCardinality'}, where
%                `[m,n]=size(S)`;
%                0 - Default, normal LP
%                'one' Minimise the Taxicab Norm using LP.
%
%                .. math::
%
%                \min \quad g_0 + \sum |v_i|
%                \text{s.t.} \quad S v = b
%                \quad \quad c^T v = f
%                \quad \quad lb \leq v \leq ub
%
%                A LP solver is required.
%
%                'zero' Minimize the cardinality (zero-norm) of v
%
%                .. math::
%
%                \min \quad d \cdot ||v||_0
%                \text{s.t.} \quad S v = b
%                \quad \quad c^T v = f
%                \quad \quad lb \leq v \leq ub
%
%                The zero-norm is approximated by a non-convex approximation
%                Six approximations are available: capped-L1 norm, exponential
%                logarithmic function, SCAD function, L_p norm with p<0, L_p norm with
%                0<p<1
%                Note : capped-L1, exponential and logarithmic function often give
%                the best result in term of sparsity.
%
%                .. See "Le Thi et al., DC approximation approaches for sparse
%                optimization,
%                European Journal of Operational Research, 2014"
%                http://dx.doi.org/10.1016/j.ejor.2014.11.031
%                A LP solver is required.
%
%                'optimizeCardinality' as for 'zero' option but uses
%                model.g0 - `n x 1` weights on zero norm, where positive is minimisation,
%                negative is maximisation, zero is neither.

```

```

%
%           The remaining options work only with a valid QP solver:
%
%           > 0   Minimises the squared Euclidean Norm of internal fluxes.
%           Typically 1e-6 works well.
GNU                                           nano                                           7.2
optimizeCbModel.m
%           .. math::
%
%           min ~& 1/2 v'*v \
%           s.t. ~& S v = b \
%           ~& c^T v = f \
%           ~& lb \leq v \leq ub
%
%           `n` x 1   Forms the diagonal of positive definite
%           matrix `F` in the quadratic program
%
%           .. math::
%
%           min ~& 0.5 v^T F v \
%           s.t. ~& S v = b \
%           ~& c^T v = f \
%           ~& lb \leq v \leq ub
%
%
%   allowLoops:    {0,(1)} If false, then instead of a conventional FBA,
%                   the solver will run an MILP version which does not allow
%                   loops in the final solution. Default is true.
%                   Runs much slower when set to false.
%                   See `addLoopLawConstraints.m` to for more info.
%
%   param:         parameters structure passed directly to solver
%                   The following are some optional fields (amongst many others)
%   *.zeroNormApprox:  approximation type of zero-norm (only available when
minNorm='zero') (default = 'cappedL1')
%
%   * 'cappedL1' : Capped-L1 norm
%   * 'exp'      : Exponential function
%   * 'log'      : Logarithmic function
%   * 'SCAD'     : SCAD function
%   * 'lp-'      : L_p norm with p<0
%   * 'lp+'      : L_p norm with 0<p<1
%   * 'all'      : try all approximations and return the best result

```

```

%
% *.verify:    verify that the input fields are consistent (default: false);
%
% OUTPUT:
% solution:    solution object:
%              * obj - Primal objective value (sum of f0,f1,f2 terms, ignoring NaN)
%              * f - Linear objective value (from LP problem)
%              * f0 - Zero-norm objective value
%              * f1 - Linear part of objective value (c*v or ||v||_1)
%              * f2 - Quadratic part of objective value
%              * v - Reaction rates (Optimal primal variable, legacy FBA solution.x)
%              * y - Dual to the matrix inequality constraints (Shadow prices)
%              * w - Dual to the box constraints (Reduced costs)
%              * s - Slacks of the metabolites
%
%              * stat - Solver status in standardized form:
%                  * 0 - Infeasible problem
%                  * 1 - Optimal solution
%                  * 2 - Unbounded solution
%                  * 3 - Almost optimal solution
%                  * -1 - Some other problem (timelimit, numerical problem etc)
%
%              * origStat - Original status returned by the specific solver
%
%              If the input model contains `C` the following fields are added to the
solution:
%
%              * ctrs_y - the duals for the constraints from C
%              * ctrs_s - Slacks of the additional constraints
%
%              If the model contains the `E` field, the following fields are added to the
solution:
%
%              * vars_v - The optimal primal values of the variables
%              * vars_w - The reduced costs of the additional variables from E
%
% .. Author:
%   - Markus Herrgard      9/16/03
%   - Ronan Fleming        4/25/09 Option to minimise the Euclidean Norm of internal
%                               fluxes using 'cplex_direct' solver
%   - Ronan Fleming        7/27/09 Return an error if any inputs are NaNp
%   - Ronan Fleming        10/24/09 Fixed 'E' for all equality constraints
%   - Jan Schellenberger   MILP option to remove flux around loops
%   - Ronan Fleming        12/07/09 Reworked minNorm parameter option to allow

```



```

%           the full range of approaches for getting
%           rid of net flux around loops.
%   - Jan Schellenberger   2/3/09   fixed bug with .f being set incorrectly
%                           when minNorm was set.
%   - Nathan Lewis         12/2/10   Modified code to allow for inequality
%                           constraints.
%   - Ronan Fleming         12/03/10   Minor changes to the internal handling of
%                           global param.
%   - Ronan Fleming         14/09/11   Fixed bug in minNorm with negative
%                           coefficient in objective
%   - Minh Le               11/02/16   Option to minimise the cardinality of
%                           fluxes vector
%   - Stefania Magnusdottir 06/02/17   Replace LPproblem2 upper bound 10000 with Inf
%   - Ronan Fleming         13/06/17   Support for coupling  $C \cdot v \leq d$ 
%   - Ronan Fleming         31/10/20   Support for optimizeCardinality.m
%
% NOTE:
%
%   `solution.stat` is either 1, 2, 3, 0 or -1, and is a translation from `solution.origStat`,
%   which is returned by each solver in a solver specific way. That is, not all solvers
%   return
%   the same type of `solution.origStat` and because the cobra toolbox can use many
%   solvers,
%   we need to return to the user of `optimizeCbModel.m` a standard representation,
%   which is what
%   `solution.stat` is.
%
%   If `solution.stat = 1 or = 3`, then a solution is returned, otherwise no solution is
%   returned
%   and the solution.f = NaN
%   This means that it is up to the person calling `optimizeCbModel` to adapt their code
%   to the
%   case when no solution is returned, by checking the value of `solution.stat` first.

% Process arguments and set up problem

% Figure out linear objective sense
if exist('osenseStr', 'var')
    if isempty(osenseStr)
        model.osenseStr = 'max';
    else
        %second argument may be the parameter structure
        if isstruct(osenseStr)

```

```

    if exist('param','var')
        error('osenseStr is a structure and param structure is also present')
    else
        %second argument is be the parameter structure
        param = osenseStr;
        if isfield(param,'osenseStr') && ~isfield(model, 'osenseStr')
            model.osenseStr = param.osenseStr;
        else
            model.osenseStr = 'max';
        end
        if isfield(param,'minNorm')
            minNorm=param.minNorm;
        else
            minNorm=[];
        end
        if isfield(param,'allowLoops')
            allowLoops=param.allowLoops;
        else
            allowLoops=1;
        end
    end
else
    % Handle osenseStr when it's a string (e.g., 'min' or 'max')
    model.osenseStr = osenseStr;
end
end
% % override if osenseStr already in the model
% if isfield(model, 'osenseStr')
%     model.osenseStr = model.osenseStr;
% else
%     model.osenseStr = 'max';
% end
end

if ~exist('param','var')
    param = struct;
end

if exist('minNorm', 'var')
    %backward compatible with minNorm true/false
    if islogical(minNorm)
        if minNorm == true
            minNorm = 1e-6;
        else

```

```

        minNorm = [];
    end
end

if isequal(minNorm,0)
    %replace minNorm = 0 with minNorm = [] to make a clear distinction
    minNorm = [];
end
% if minNorm = 'zero' then check the parameter 'zeroNormApprox'
if isequal(minNorm,'zero')
    if isfield(param,'zeroNormApprox')
        zeroNormApprox = param.zeroNormApprox;
        availableApprox = {'cappedL1','exp','log','SCAD','lp-','lp+','all'};
        if ~ismember(zeroNormApprox,availableApprox)
            warning('Approximation is not valid. Default value will be used');
            zeroNormApprox = 'cappedL1';
        end
    else
        zeroNormApprox = 'cappedL1';
    end
end
else
    %use global solver parameter for minNorm
    minNorm = getCobraSolverParams('LP','minNorm');
end
if exist('allowLoops', 'var')
    if isempty(allowLoops)
        allowLoops = true;
    end
else
    allowLoops = true;
end

%use global solver parameter, unless these these are specified in the input
[printLevel,          primalOnlyFlag,          verify,feasTol]      =
getCobraSolverParams('LP',{'printLevel','primalOnly', 'verify','feasTol'},param);

% size of the stoichiometric matrix
[nMets,nRxns] = size(model.S);

if isfield(model,'C')
    modelC = 1;
    nCtrs = size(model.C,1);
else

```

```
    modelC = 0;  
    nCtrs = 0;  
end
```

```
if isfield(model,'E')  
    modelE = 1;  
    nVars = size(model.E,2);  
else  
    modelE = 0;  
    nVars = 0;  
end
```

```
if ischar(minNorm)  
    if strcmp(minNorm, 'oneInternal')  
        SConsistentRxnBool=model.SConsistentRxnBool;  
    end  
end
```

```
if isfield(model,'g') && ~isfield(model,'cf')%in case it is a thermodynamic model  
    if isfield(model,'g0') || isfield(model,'g1')  
        warning('model.g ignored by optimizeCbModel. zero and one norm weights are  
separately specified in model.g0 and model.g1 respectively')  
    else  
        error('model.g no longer supported by optimizeCbModel. zero and one norm weights  
must be separately specified in model.g0 and model.g1 respectively')  
    end  
end
```

```
%weights on zero norm  
if isfield(model,'g0')  
    if length(model.g0)~=nRxns  
        error('model.g0 must be nRxns x 1')  
    end  
    zeroNormWeights=columnVector(model.g0);  
    if size(zeroNormWeights,2)~=1  
        error('model.g0 must be nRxns x 1')  
    end  
else  
    zeroNormWeights=[];  
end
```

```
%weights on one norm  
if isfield(model,'g1')
```

```

if length(model.g1)~=nRxns
    error('model.g1 must be nRxns x 1')
end
oneNormWeights=columnVector(model.g1);
if size(oneNormWeights,2)~=1
    error('model.g1 must be nRxns x 1')
end
else
    oneNormWeights=[];
end

if ~isempty(zeroNormWeights)
    if all(zeroNormWeights==1) && ischar(minNorm)
        if strcmp('minNorm','optimizeCardinality') && ~isempty(oneNormWeights)
            %no need to use optimize cardinality if effectively only
            %minimisation of one norm is being requested
            minNorm = 'one';
        end
    end
end

%weights on two norm
if isfield(model,'g2')
    error('minimisation of two norm in combination with one and zero norm is not yet
supported')
    if length(model.g2)~=nRxns
        error('model.g2 must be nRxns x 1')
    end
    twoNormWeights=columnVector(model.g2);
    if size(twoNormWeights,2)~=1
        error('model.g2 must be nRxns x 1')
    end
else
    twoNormWeights=[];
end

%by default, do linear optimisation unless not required
doLinearOptimisationFirst = 1;
%if there is no linear objective, do a QP
if all(model.c==0)
    doLinearOptimisationFirst = 0;
end

% if there is no linear objective and no quadratic objective, do an LP

```

```

if isempty(minNorm)
    doLinearOptimisationFirst = 1;
end

% If there is are linear and quadrative objectives but the bounds on the
% corresponding reaction are fixed, then there is no need to solve an LP
% first, so do a QP
if all( (model.lb == model.ub & model.c~=0) == (model.c~=0)) && ~isempty(minNorm)
    doLinearOptimisationFirst = 0;
end

% If this is a quadratically regularised LP, go straight to QP
% TODO This is a hack of the param.minNorm to direct solution to QRLP or QRQP
if isfield(param,'solveWBMmethod')
    if any(strcmp(param.solveWBMmethod,{'QP','QRLP','QRQP'}))%TODO 'QRLP','QRQP'
        need coded in
        model.c(:)=0;
        doLinearOptimisationFirst = 0;
        minNorm = param.minNorm;
    end
end

% build the optimization problem
optProblem = buildOptProblemFromModel(model,verify,param);
% save the original size of the problem
[~,nTotalVars] = size(optProblem.A); % nTotalVars needed even if optProblem not used
for an LP

%%
t1 = clock;
if doLinearOptimisationFirst
    if allowLoops && ~strcmp(minNorm,'optimizeCardinality')
        clear model
    end

    if 0
        %debug
        solution=solveCobraLPCPLEX(optProblem,1,0,0,[],0,'ILOGcomplex');
        solution.f=solution.obj;
        return
    end
end

```

```

% Solve initial LP
if allowLoops
    paramLP = param;
    if isfield(paramLP,'minNorm')
        paramLP = rmfield(paramLP,'minNorm');
    end
    solution = solveCobraLP(optProblem, paramLP);
else
    MILPproblem = addLoopLawConstraints(optProblem, model, 1:nRxns);
    solution = solveCobraMILP(MILPproblem);
end

%save objective from LP
objectiveLP = solution.obj;

if strcmp(solution.solver,'mps')
    return;
end
else
    %no need to solve an LP first
    objectiveLP = [];
end

%only run if minNorm is not empty, and either there is no linear objective
%or there is a linear objective and the LP problem solved to optimality
if (doLinearOptimisationFirst==0 && ~isempty(minNorm)) ||
(doLinearOptimisationFirst==1 && solution.stat==1 && ~isempty(minNorm))

if strcmp(minNorm, 'optimizeCardinality')
    % DC programming for solving the cardinality optimization problem
    % The `l0` norm is approximated by a capped-`l1` function.
    %
    % :math:`\min c'(x, y, z) + \lambda_0 k. ||x||_0 + \lambda_1 ||x||_1`
    % . - \delta_0 d. ||y||_0 + \delta_1 ||y||_1`
    % s.t. :math:`A*(x, y, z) <= b`
    % :math:`l <= (x,y,z) <= u`
    % :math:`x \in R^p, y \in R^q, z \in R^r`
    %
    % USAGE:
    %
    % solution = optimizeCardinality(problem, param)

    % * .lambda0 - trade-off parameter on minimise `||x||_0`

```

```

if isfield(model,'lambda0')
    optProblem.lambda0=model.lambda0;
end
% * .lambda1 - trade-off parameter on minimise `||x||_1`
if isfield(model,'lambda1')
    optProblem.lambda1=model.lambda1;
end
% * .delta0 - trade-off parameter on maximise `||y||_0`
if isfield(model,'delta0')
    optProblem.delta0=model.delta0;
end
% * .delta1 - trade-off parameter on minimise `||y||_1`
if isfield(model,'delta1')
    optProblem.delta1=model.delta1;
end

if isfield(optProblem,'F')
    error('optimizeCardinality does not (yet) support minimisation of 2-norm')
end

if any(oneNormWeights<0)
    error('optimizeCardinality does not (yet) support maximisation of 1-norm')
end

% * .p - size of vector `x` OR a `size(A,2) x 1` boolean indicating columns of A
corresponding to x (min zero norm).
optProblem.p = zeroNormWeights > 0;
% * .q - size of vector `y` OR a `size(A,2) x 1` boolean indicating columns of A
corresponding to y (max zero norm).
optProblem.q = zeroNormWeights < 0;
% * .r - size of vector `z` OR a `size(A,2) x 1` boolean indicating columns of A
corresponding to z .
optProblem.r = zeroNormWeights == 0;

% problem: Structure containing the following fields describing the problem:
% * .k - `p x 1` OR a `size(A,2) x 1` strictly positive weight vector on minimise
`||x||_0`
if isempty(zeroNormWeights)
    error('optimizeCardinality expects weights on zero norm, but model.g0 is empty.')
end
k = zeroNormWeights;
k(k<0)=0;
optProblem.k = k;

```



```

%      * .d - `q x 1` OR a `size(A,2) x 1` strictly positive weight vector on maximise
`||y||_0`
d = -zeroNormWeights;
d(d<0)=0;
optProblem.d = d;
%      * .o `size(A,2) x 1` strictly positive weight vector on minimise `||[x;y;z]||_1`
optProblem.o = oneNormWeights;

% param:   Parameters structure:
%          * .printLevel - greater than zero to receive more output
% The following use default values, unless they are provided in the
% param structure
%          * .nbMaxIteration - stopping criteria - number maximal of iteration
(Default value = 100)
%          * .epsilon - stopping criteria - (Default value = 1e-6)
%          * .theta - starting parameter of the approximation (Default value = 0.5)
%          For a sufficiently large parameter  $\wedge R$ , the Capped-L1 approximate
problem
%          and the original cardinality optimisation problem are have the
same set of optimal solutions
%          * .thetaMultiplier - at each iteration: theta = theta*thetaMultiplier
%          * .eta - Smallest value considered non-zero (Default value feasTol*1000)

if doLinearOptimisationFirst
    optProblem2 = optProblem;
    optProblem2.A = [optProblem.A ; optProblem.c'];
    optProblem2.b = [optProblem.b ; objectiveLP];
    optProblem2.csense = [optProblem.csense;'E'];
    optProblem2.lb = optProblem.lb;
    optProblem2.ub = optProblem.ub;
    solCard = optimizeCardinality(optProblem2, param);
else
    % The following are assumed to be inherited correctly from
    % optProblem built above
    %      * .A - `s x size(A,2)` LHS matrix
    %      * .b - `s x 1` RHS vector
    %      * .lb - `size(A,2) x 1` Lower bound vector
    %      * .ub - `size(A,2) x 1` Upper bound vector
    %      * .c - `size(A,2) x 1` linear objective function vector
    %      * .osense - Objective sense for problem.c only (1 means minimise (default),
-1 means maximise)
    %      * .csense - `s x 1` Constraint senses, a string containing the constraint sense
for
    %          each row in `A` ('E', equality, 'G' greater than, 'L' less than).

```

```

        solCard = optimizeCardinality(optProblem, param);
    end

    solution.stat = solCard.stat;
    solution.full = solCard.xyz;
    solution.dual = [];
    solution.rcost = [];
    solution.slack = [];

elseif strcmp(minNorm, 'zero')
    % Minimize the cardinality (zero-norm) of v
    %      min ||v||_0
    %      s.t.   S*v = b
    %            c*v = f
    %            lb <= v <= ub

    % Define the constraints structure
    if doLinearOptimisationFirst
        optProblem2.A = [optProblem.A ; optProblem.c'];
        optProblem2.b = [optProblem.b ; objectiveLP];
        optProblem2.csense = [optProblem.csense;'E'];
        optProblem2.lb = optProblem.lb;
        optProblem2.ub = optProblem.ub;
        % Call the sparse LP solver
        solutionL0 = sparseLP(optProblem2, zeroNormApprox);
    else
        % Call the sparse LP solver
        solutionL0 = sparseLP(optProblem, zeroNormApprox);
    end

    %Store results
    solution.stat = solutionL0.stat;
    solution.full = solutionL0.x;
    solution.dual = [];
    solution.rcost = [];
    solution.slack = [];

elseif strcmp(minNorm, 'one')
    % Minimise the absolute value of fluxes
    % Solve secondary LP to optimize weighted 1-norm of v
    % Weight provided by model.g1
    % Set up the optimization problem
    % min model.g1*(vf + vr)
    % 1: S*vf -S*vr = b

```

```

% 3: vf >= -v
% 4: vr >= v
% 5: c'v >= f or c'v <= f (optimal value of objectiveLP)
%
% vf,vr >= 0

optProblem2.A = [optProblem.A sparse(nMets+nCtrs,2*nRxns);
    speye(nRxns,nTotalVars) speye(nRxns,nRxns) sparse(nRxns,nRxns);
    -speye(nRxns,nTotalVars) sparse(nRxns,nRxns) speye(nRxns,nRxns);
    optProblem.c' sparse(1,2*nRxns)];

if ~isempty(oneNormWeights)
    %weighted one norm
    optProblem2.c =
[zeros(nTotalVars,1);[oneNormWeights;oneNormWeights].*ones(2*nRxns,1)];
    else
        optProblem2.c = [zeros(nTotalVars,1);ones(2*nRxns,1)];
    end
    optProblem2.lb = [optProblem.lb;zeros(2*nRxns,1)];
    optProblem2.ub = [optProblem.ub;Inf*ones(2*nRxns,1)];
    if isempty(objectiveLP)
        objectiveLP = 0;
    end
    optProblem2.b = [optProblem.b;zeros(2*nRxns,1);objectiveLP];

    optProblem2.b = [optProblem.b;zeros(2*nRxns,1);objectiveLP];

%csense for 3 & 4 above
optProblem2.csense = [optProblem.csense; repmat('G',2*nRxns,1)];
% constrain the optimal value according to the original problem
if optProblem.osense==-1
    optProblem2.csense = [optProblem2.csense; 'G'];
    %LPproblem2.csense(nTotalVars+1) = 'G'; %wrong
else
    optProblem2.csense = [optProblem2.csense; 'L'];
    %LPproblem2.csense(nTotalVars+1) = 'L'; %wrong
end

optProblem2.osense = 1;%minimise
% Re-solve the problem
if allowLoops
    solution = solveCobraLP(optProblem2, param);
else
    MILPproblem2 = addLoopLawConstraints(optProblem2, model, 1:nRxns);

```

```

        solution = solveCobraMILP(MILPproblem2);
    end
elseif strcmp(minNorm, 'oneInternal')
    % Minimize the absolute value of internal fluxes to eliminate
    % thermodynamically infeasible solutions
    % CycleFreeFlux: efficient removal of thermodynamically infeasible loops from flux
    distributions
    % Desouki et al Bioinformatics, Volume 31, Issue 13, 1 July 2015, Pages 2159–2165,
    https://doi.org/10.1093/bioinformatics/btv096
    %
    % Solve secondary LP to minimise weighted 1-norm of v
    % Set up the optimization problem
    % min model.g1*(p + q)
    % 1:  $S \cdot v_1 = b$ 
    % 3:  $v_1 - p + q = 0$ 
    % 4:  $c \cdot v_1 \geq f$  or  $c \cdot v_1 \leq f$  (optimal value of objectiveLP)
    % 5:  $p, q \geq 0$ 

    nIntRxns=nnz(SConsistentRxnBool);
    A2 = sparse(nIntRxns,nTotalVars);
    A2(:,SConsistentRxnBool)=speye(nIntRxns,nIntRxns);
    optProblem2.A = [...
        optProblem.A,          sparse(nMets,2*nIntRxns);
        A2, -speye(nIntRxns,nIntRxns), speye(nIntRxns,nIntRxns);
        optProblem.c',          sparse(1,2*nIntRxns)];

    %only minimise the absolute value of internal reactions
    if ~isempty(oneNormWeights)
        %only the weights on the internal reactions have an effect, the
        %rest are discarded
        oneNormWeightsInt=oneNormWeights(SConsistentRxnBool);
        if any(oneNormWeightsInt<0)
            warning('minNorm = "oneInternal" may not eliminate thermodynamically
            infeasible fluxes if model.g1(SConsistentRxnBool) entries are negative')
        end
        %weighted one norm of internal reactions

        optProblem2.c =
[zeros(nTotalVars,1);[oneNormWeightsInt;oneNormWeightsInt].*ones(2*nIntRxns,1)];
    else
        optProblem2.c = [zeros(nTotalVars,1);ones(2*nIntRxns,1)];
    end
    optProblem2.lb = [optProblem.lb;zeros(2*nIntRxns,1)];
    optProblem2.ub = [optProblem.ub;Inf*ones(2*nIntRxns,1)];
    optProblem2.b = [optProblem.b;zeros(nIntRxns,1);objectiveLP];

```

```

%csense for 3 above
optProblem2.csense = [optProblem.csense; repmat('E',nIntRxns,1)];
% constrain the optimal value according to the original problem
if optProblem.osense==-1
    optProblem2.csense = [optProblem2.csense; 'G'];
else
    optProblem2.csense = [optProblem2.csense; 'L'];
end
%minimise absolute value of internal reaction fluxes
optProblem2.osense = 1;

% Re-solve the problem
solution = solveCobraLP(optProblem2, param);

elseif strcmp(minNorm, 'QRLP')
    buildOptProblemFromModel_param = param;
    buildOptProblemFromModel_param.minNorm = param.minNormWBM;
    optProblem = buildOptProblemFromModel(model, 0,
    buildOptProblemFromModel_param);
    solutionQRLP = solveCobraQP(optProblem,param);

    solution.full = solutionQRLP.full(1:optProblem.n);% Full QP solution vector
    solution.rcost = solutionQRLP.rcost(1:optProblem.n);% Reduced costs, dual
    solution to :math:`lb \leq x \leq ub`
    solution.dual = solutionQRLP.dual(1:optProblem.m);% dual solution to
    :math:`A*x \leq/= \geq b`
    solution.slack = solutionQRLP.full(1:optProblem.m);% slack variable such that
    :math:`A*x + s = b`
    solution.obj = model.c*solution.full;% Objective value
    solution.solver = solutionQRLP.solver;% Solver used to solve QP problem
    solution.origStat = solutionQRLP.origStat;% Original status returned by the
    specific solver
    solution.time = solutionQRLP.time;% Solve time in seconds
    solution.stat = solutionQRLP.stat;% Solver status in standardized form (see
    below)
    solution.r = solutionQRLP.full(optProblem.n+1:optProblem.m); %  $A*x + r \leq= b$ 
    solution.p =
    solutionQRLP.full(optProblem.n+optProblem.m+1:optProblem.n+optProblem.m+optProbl
    em.n);
    solution.p(solution.p < feasTol) = 0; %  $lb + p \leq x \leq ub + q$ 
    solution.q =
    solutionQRLP.full(optProblem.n+optProblem.m+1:optProblem.n+optProblem.m+optProbl
    em.n); %  $lb + p \leq x \leq ub + q$ 

```

```

solution.q(solution.q > -feasTol) = 0; % lb + p <= x <= ub + q
solution.q = -solution.q; % lb + p <= x <= ub + q

elseif strcmp(minNorm, 'QRQP')
    minNorm = minNormWBM;
    optProblem = buildOptProblemFromModel(model, 0, param);
    solution = solveCobraQP(optProblem);

elseif length(minNorm)> 1 || minNorm > 0
    %THIS SECTION BELOW ASSUMES WRONGLY THAT c HAVE ONLY ONE NONZERO
    SO I
    %REPLACED IT WITH A MORE GENERAL FORMULATION, WHICH IS ALSO ROBUST
    TO
    %THE CASE WHEN THE OPTIMAL OBJECIVE WAS ZERO - RONAN June 13th 2017
    % if nnz(optProblem.c)>1
    %     error('Code assumes only one non-negative coefficient in linear
    %         part of objectiveLP');
    % end
    % % quadratic minimization of the norm.
    % % set previous optimum as constraint.
    % optProblem.A = [optProblem.A;
    %     (optProblem.c~=0 + 0)];%new constraint must be a row with a single unit
    entry
    % optProblem.csense(end+1) = 'E';
    %
    % optProblem.b = [optProblem.b;solution.full(optProblem.c~=0)];

    %Minimise Euclidean norm using quadratic programming
    if isnumeric(minNorm)
        if length(minNorm)==nTotalVars && size(minNorm,1)~=size(minNorm,2)
            minNorm=columnVector(minNorm);
        elseif isscalar(minNorm)
            minNorm=ones(nTotalVars,1)*minNorm;
        else
            error(['minNorm has dimensions ' int2str(size(minNorm,1)) ' x '
int2str(size(minNorm,2)) ' but it can only of the form {(0), "one", "zero", > 0 , n x 1
vector}.'])
        end
    elseif ischar(minNorm) && length(minNorm)==4 && strcmp(minNorm,'1e-6')
        %handle the aberrant case when minNorm is provided as a string
        minNorm=1e-6;
        minNorm=ones(nTotalVars,1)*minNorm;
    else

```

```

        error(['minNorm has dimensions ' int2str(size(minNorm,1)) ' x '
int2str(size(minNorm,2)) ' but it can only of the form {(0), "one", "zero", > 0 , n x 1
vector}.'.])

```

```

    end

```

```

    % quadratic minimization of the norm.

```

```

    if doLinearOptimisationFirst

```

```

        % set previous linear optimum as constraint.

```

```

        optProblem2 = optProblem;

```

```

        optProblem2.A = [optProblem.A;optProblem.c'];

```

```

        optProblem2.b = [optProblem.b;objectiveLP];

```

```

        optProblem2.csense = [optProblem.csense; 'E'];

```

```

        optProblem2.F = spdiags(minNorm,0,nTotalVars,nTotalVars);

```

```

        optProblem2.osense=1;

```

```

        if allowLoops

```

```

            %quadratic optimization will get rid of the loops unless you are maximizing a
flux which is

```

```

            %part of a loop. By definition, exchange reactions are not part of these loops,
more

```

```

            %properly called stoichiometrically balanced cycles.

```

```

            solution = solveCobraQP(optProblem2,param);

```

```

        else

```

```

            %this is slow, but more useful than minimizing the Euclidean norm if one is
trying to

```

```

            %maximize the flux through a reaction in a loop. e.g. in flux variability analysis

```

```

            MIQPproblem = addLoopLawConstraints(optProblem2, model, 1:nTotalVars);

```

```

            solution = solveCobraMIQP(MIQPproblem);

```

```

        end

```

```

    else

```

```

        optProblem.F = spdiags(minNorm,0,nTotalVars,nTotalVars);

```

```

        if allowLoops

```

```

            %quadratic optimization will get rid of the loops unless you are maximizing a
flux which is

```

```

            %part of a loop. By definition, exchange reactions are not part of these loops,
more

```

```

            %properly called stoichiometrically balanced cycles.

```

```

            solution = solveCobraQP(optProblem,param);

```

```

        else

```

```

            %this is slow, but more useful than minimizing the Euclidean norm if one is
trying to

```

```

            %maximize the flux through a reaction in a loop. e.g. in flux variability analysis

```

```

            MIQPproblem = addLoopLawConstraints(optProblem, model, 1:nTotalVars);

```

```

            solution = solveCobraMIQP(MIQPproblem);

```

```

        end
    end
end
end

% %TODO fix this Hack in case param.minNorm is used again
% if ~isempty(param.solveWBMmethod)
%     param.minNorm = param.minNormWBM;
% end

%dummy parts of the solution
solution.f0 = NaN;
solution.f1 = NaN;
solution.f2 = NaN;

switch solution.stat
    case 1
        if printLevel>0
            fprintf('%s\n','Optimal solution found.')
        end
    case -1
        if printLevel>0
            warning('%s\n','No solution reported (timelimit, numerical problem etc).')
        end
    case 0
        if printLevel>0
            warning('Infeasible model.')
        end
    case 2
        if printLevel>0
            warning('Unbounded model.');

```


end

% Return a solution or an almost optimal solution

if solution.stat == 1 || solution.stat == 3

 % solution found. Set corresponding values

 %the value of the linear part of the objective is always the optimal objective from the first LP

 if isempty(objectiveLP)

 solution.f = objectiveLP;

 else

 solution.f = optProblem.c'*solution.full(1:nTotalVars,1);

 end

if isempty(minNorm)

 minNorm = 'empty';

end

if isnumeric(minNorm)

 minNorm = 'two';

end

%the value of the second part of the objective depends on the norm

switch minNorm

 case 'empty'

 solution.f0 = NaN;

 solution.f1 = optProblem.c'*solution.full(1:nTotalVars,1);

 solution.f2 = NaN;

 case 'zero'

 %zero norm

 solution.f0 = sum(abs(solution.full(1:nTotalVars,1)) > feasTol);

 solution.f1 = NaN;

 solution.f2 = NaN;

 case 'one'

 solution.f0 = NaN;

 %one norm

 solution.f1 = sum(abs(solution.full(1:nTotalVars,1)));

 solution.f2 = NaN;

 case 'two'

 solution.f0 = NaN;

 if isfield(optProblem,'c')

 solution.f1 = optProblem.c'*solution.full(1:nTotalVars,1);

 if isfield(solution,'objLinear')

 solution = rmfield(solution,'objLinear');

 end

 else

```

        solution.f1 = NaN;
    end
    if isfield(optProblem,'F')
        solution.f2 = 0.5*solution.full*optProblem.F*solution.full;
    else
        disp(param)
        warning('optProblem.F missing')
    end
    if isfield(solution,'objQuadratic')
        solution.f2 = solution.objQuadratic;
        solution = rmfield(solution,'objQuadratic');
    end
    otherwise
        if exist('LPproblem2','var')
            if isfield(optProblem2,'F')
                solution.f0 = NaN;
                solution.f1 = optProblem.c*solution.full(1:nTotalVars,1);
                solution.f2 = 0.5*solution.full*optProblem2.F*solution.full;
            end
        else
            solution.f0 = NaN;
            solution.f1 = NaN;
            solution.f2 = NaN;
        end
    end
    solution.obj = sum([solution.f0,solution.f1,solution.f2],'omitnan');

%primal optimal variables
solution.v = solution.full(1:nRxns);
if modelE
    solution.vars_v = solution.full(nRxns+1:nRxns+nVars);
else
    solution.vars_v = [];
end
%provided for backward compatibility
solution.x = solution.v;

% handle the duals, reducing them to fields in the model.
if isfield(solution,'dual')
    if ~isempty(solution.dual)
        solution.y = solution.dual(1:nMets,1);
        if modelC
            solution.ctrs_y = solution.dual(nMets+1:nMets+nCtrs,1);
        end
    end
end

```

```

    end
end

% handle reduced costs
if isfield(solution,'rcost')
    if ~isempty(solution.rcost)
        solution.w=solution.rcost(1:nRxns,1);
        if modelE
            solution.vars_w = solution.rcost(nRxns+1:nRxns+nVars,1);
        end
    end
end
end

% handle slacks
if isfield(solution,'slack')
    if ~isempty(solution.slack)
        solution.s=solution.slack(1:nMets,1);
        if modelC
            solution.ctrs_s = solution.slack(nMets+1:nMets+nCtrs,1);
        end
    end
end
end

%if (~primalOnlyFlag && allowLoops && any(~minNorm)) % LP rcost/dual only correct
if not doing minNorm
    % LP rcost/dual are still meaningful if doing, one simply has to be aware that there is a
    % perturbation to them the magnitude of which depends on norm(minNorm) - Ronan
    if (~primalOnlyFlag && allowLoops)
        if ~isempty(solution.dual)
            solution.y = solution.dual(1:nMets,1);
        end
        if modelC
            solution.ctrs_y = solution.dual(nMets+1:nMets+nCtrs,1);
        end
        solution.w = solution.rcost;
        solution.s = solution.slack;
    end

    solution.time = etime(clock, t1);

    fieldOrder =
{'f';'f0';'f1';'f2';'v';'y';'w';'s';'solver';'lpmethod';'qpmethod';'stat';'origStat';'time';'basis';'var
s_v';'vars_w';'ctrs_y';'ctrs_s';'x';'full';'obj';'rcost';'dual';'slack'};
    % reorder fields for better readability

```

```

currentfields = fieldnames(solution);
presentfields = ismember(fieldOrder,currentfields);
absentfields = ~ismember(currentfields,fieldOrder);
solution = orderfields(solution,[currentfields(absentfields);fieldOrder(presentfields)]);
else
    if 0
        %return NaN of correct dimensions if problem does not solve properly
        solution.f = NaN;

        solution.v = NaN*ones(nRxns,1);
        solution.y = NaN*ones(nMets,1);
        solution.w = NaN*ones(nRxns,1);
        solution.s = NaN*ones(nMets,1);
        if modelC
            solution.ctr_s_y = NaN*ones(nCtrs,1);
            solution.ctr_s_s = NaN*ones(nCtrs,1);
        end
        if modelE
            solution.vars_v = NaN*ones(nVars,1);
            solution.vars_w = NaN*ones(nVars,1);
        end
    end
else
    %return empty fields if problem does not solve properly (backward
    %compatible)
    solution.obj = NaN;
    solution.f = NaN;
    solution.f0 = NaN;
    solution.f1 = NaN;
    solution.f2 = NaN;
    solution.v = [];
    solution.y = [];
    solution.w = [];
    solution.s = [];
    if modelC
        solution.ctr_s_y = [];
        solution.ctr_s_s = [];
    end
    if modelE
        solution.vars_v = [];
        solution.vars_w = [];
    end
end
end
solution.x = solution.v;
solution.time = etime(clock, t1);

```

end

```
if 1 %this may not be very backward compatible  
    %remove fields coming from solveCobraLP/QP but not part of the specification  
    %of the output from optimizeCbModel  
    if isfield(solution,'full')  
        solution = rmfield(solution,'full');  
    end  
    if isfield(solution,'dual')  
        solution = rmfield(solution,'dual');  
    end  
    if isfield(solution,'rcost')  
        solution = rmfield(solution,'rcost');  
    end  
    if isfield(solution,'slack')  
        solution = rmfield(solution,'slack');  
    end  
end
```

El archivo **saveTableAsCSV.m** (es una modificación del archivo **writetable.m** para que funcionase en **MATLAB r2022a**) contiene:

```
function saveTableAsCSV(tbl, filename)  
    % ABSOLUTE_SIMPLE_TABLE2CSV - Guaranteed working version for R2022a  
  
    % Open file  
    fid = fopen(filename, 'w');  
    if fid == -1  
        error('Cannot open file: %s', filename);  
    end  
  
    % Write header  
    fprintf(fid, '%s\n', strjoin(tbl.Properties.VariableNames, ','));  
  
    % Convert entire table to cell array of strings  
    strData = cell(size(tbl));  
    for col = 1:width(tbl)  
        for row = 1:height(tbl)  
            try  
                % Convert every value to string in simplest way possible  
                strData{row,col} = char(string(tbl{row,col}));  
            catch  
                strData{row,col} = '';  
            end  
        end  
    end
```

```

        end
    end

    % Write each row
    for row = 1:size(strData,1)
        % Simple CSV formatting - just escape quotes
        rowStr = strData(row,:);
        rowStr = strrep(rowStr, '"', '""');
        fprintf(fid, '%s\n', strjoin(rowStr, ','));
    end

    fclose(fid);
end

```

2. Uso de Exp 2 Flux para ajustar un modelo metabólico a un archivo de expresión

Inicialmente se debe acceder al directorio que contiene el archivo **Mariabotara.m**, luego se debe ejecutar la función **Mariabotara()** en la pseudo consola de MATLAB.

NOTA: los nombres de las columnas del archivo de expresión **.csv** deben ser **Gene,Expression_Value** para que el script **Mariaexp2flux.m** identifique las columnas.

```

= Checking environment...

One subSystem per reaction, so each model.subSystems{x} is a character array.
= Model "botara_250214.xml" loaded successfully. Proceeding...
= CSV file "Clon17R2extracted_file1.csv" correctly loaded.
Checking model boundaries...
Checking C vector correctness...
Extracting genes names and expression values
Error using e2f (line 218)
Unrecognized table variable name 'Gene'.

```

Con el comando nano **Clon17R2extracted_file1.csv** se modificaron los nombres de las columnas del archivo de expresión **Clon17R2extracted_file1.csv**, el mismo proceso se repitió con los demás archivos de expresión.

```

GNU nano 7.2                               Clon17R2extracted_file1.csv
Gene,Expression_Value
DM8sca01G00010,0.000000
DM8sca01G00020,0.000000
DM8sca01G00030,0.000000
DM8sca10G00010,0.000000

```

```
(base) gibbs@gibbs:~/MATLABdocker/cobratoolbox/E2F/E2F$ wc -l Clon17ncbi6.csv
363 Clon17ncbi6.csv
(base) gibbs@gibbs:~/MATLABdocker/cobratoolbox/E2F/E2F$ wc -l Clon39ncbi6.csv
363 Clon39ncbi6.csv
(base) gibbs@gibbs:~/MATLABdocker/cobratoolbox/E2F/E2F$ wc -l ClonColombiancbi6.csv
363 ClonColombiancbi6.csv
(base) gibbs@gibbs:~/MATLABdocker/cobratoolbox/E2F/E2F$ wc -l FlorBlancancbi6.csv
363 FlorBlancancbi6.csv
```

Comandos adicionales

A. La herramienta conda se puede descargar con el comando **wget** <https://www.anaconda.com/download/success> el cual descarga un archivo ejecutable bash (.sh) capaz de instalar la herramienta al ser ejecutado. Posterior a la instalación se debe reiniciar la consola o salir y volver a entrar al servidor, pues el programa lo requiere para luego ser activado con el comando **conda activate**

```
gibbs@gibbs:~/tools$ wget https://repo.anaconda.com/archive/Anaconda3-2024.10-1-Linux-x86_64.sh
--2025-04-09 20:24:46-- https://repo.anaconda.com/archive/Anaconda3-2024.10-1-Linux-x86_64.sh
Resolving repo.anaconda.com (repo.anaconda.com)... 104.16.32.241, 104.16.191.158, 2606:4700::6810:20f1, ...
Connecting to repo.anaconda.com (repo.anaconda.com)|104.16.32.241|:443... connected.
HTTP request sent, awaiting response... 200 OK
Length: 1102495056 (1.0G) [application/octet-stream]
Saving to: 'Anaconda3-2024.10-1-Linux-x86_64.sh'
64.sh                20%[=>] 218.20M 27.9MB/s eta 31s

Please answer 'yes' or 'no':
>>> yes
Anaconda3 will now be installed into this location:
/home/gibbs/anaconda3
- Press ENTER to confirm the location
- Press CTRL-C to abort the installation
- Or specify a different location below

[/home/gibbs/anaconda3] >>>
PREFIX=/home/gibbs/anaconda3
Unpacking payload ...
```

```
gibbs@gibbs:~$ conda activate
(base) gibbs@gibbs:~$
```

B. La herramienta tmux se puede descargar e instalar usando los comandos **sudo apt update** seguido del comando **sudo apt install tmux**. Posterior a la instalación se puede activar la herramienta con el comando **tmux** y se desactiva con la combinación de teclas **CTRL+b d**. Si se quiere reabrir una sesión de tmux se puede usar el comando **tmux ls** para listar las sesiones de tmux existentes y luego usar el comando **tmux attach-session -t 0** donde lo que sigue después de la **-t** es el nombre de la sesión existente a la que se quiere acceder. Si uno quiere subir o bajar con las flechas del teclado en la sesión de tmux, primero se debe usar la combinación de teclas **ctrl+b** seguida de **alt+(** (eso en el caso de ubuntu), para salir de la modalidad de movimiento se oprime la tecla **q**.

```
(base) gibbs@gibbs:~/analysis2.0/QualityControl/RawDataQC$ sudo apt update
[sudo] password for gibbs:
```

```
(base) gibbs@gibbs:~/analysis2.0/QualityControl/RawDataQC$ sudo apt install tmux
Reading package lists... Done
```

```
(base) gibbs@gibbs:~/analysis2.0/QualityControl/RawDataQC$ tmux
[detached (from session 0)]
```

```
(base) gibbs@gibbs:~/analysis2.0/QualityControl/RawDataQC$ █
```

```
(base) gibbs@gibbs:~/analysis2.0/QualityControl/RawDataQC$ █
```

```
[0] 0:bash* "gibbs" 21:19 09-Apr-25
```

```
(base) gibbs@gibbs:~/analysis2.0/QualityControl/RawDataQC$ tmux ls
0: 1 windows (created Wed Apr 9 21:19:19 2025)
```

```
(base) gibbs@gibbs:~/analysis2.0/QualityControl/RawDataQC$ tmux attach-session -t 0
[detached (from session 0)]
```

Por otro lado, se pueden crear sesiones tmux con nombres particulares usando el comando **tmux new -s session_name** donde lo que sigue después de la **-s** es el nombre que se quiere dar a la sesión. Con el comando **tmux ls** se pueden confirmar las sesiones de tmux existentes entre las que ya se encuentra **session_name** y **0**.

```
(base) gibbs@gibbs:~/analysis2.0/QualityControl/RawDataQC$ tmux new -s session_name
[detached (from session session_name)]
```

```
(base) gibbs@gibbs:~/analysis2.0/QualityControl/RawDataQC$ tmux ls
0: 1 windows (created Wed Apr 9 21:19:19 2025)
session_name: 1 windows (created Wed Apr 9 21:36:42 2025)
```

Aunque si se quiere borrar una sesión se puede usar el comando **tmux kill-session -t session_name** donde **session_name** es el nombre de la sesión a borrar. Con el comando

tmux ls se pueden confirmar las sesiones de tmux existentes entre las que ya no se encuentra **session_name**.

```
(base) gibbs@gibbs:~/analysis2.0/QualityControl/RawDataQC$ tmux kill-session -t session_name
(base) gibbs@gibbs:~/analysis2.0/QualityControl/RawDataQC$ tmux ls
0: 1 windows (created Wed Apr 9 21:19:19 2025)
```